



计算机网络（第7版）

谢希仁 编著



电子工业出版社
PUBLISHING HOUSE OF ELECTRONICS INDUSTRY
<http://www.phei.com.cn>

第5章 运输层



5.4

可靠传输的工作原理

5.5

TCP 报文段的首部格式

5.6

TCP 可靠传输的实现



上节回顾

- 传输层功能：端到端应用进程间逻辑通信
- 传输层协议：
 - UDP：无连接、面向报文
 - TCP：面向连接、面向字节流
- 端口号与套接字
 - UDP：使用端口号
 - TCP：使用套接字（IP地址+端口号）
- UDP与IP协议
 - 相同：无连接、尽力投递
 - 差异：
 - 层次不同
 - 通信作用位置不同
 - 校验内容不同（IP：首部；UDP：首部+伪首部+数据）



5.4

可靠传输的 工作原理

5.4.1

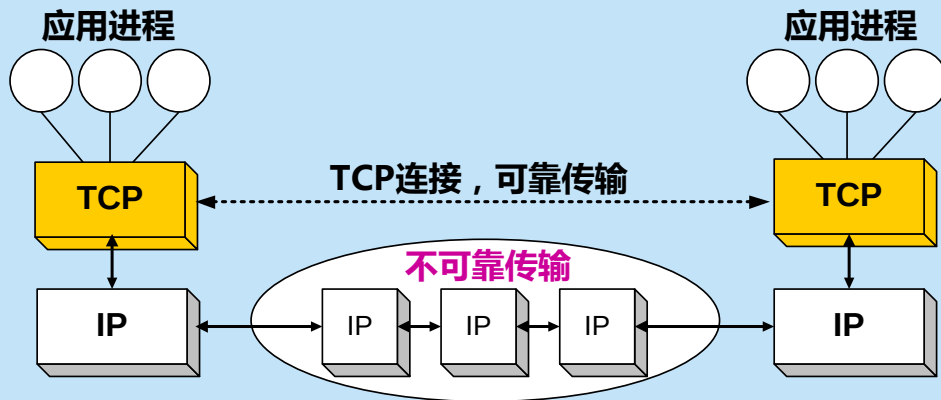
停止等待协议

5.4.2

连续 ARQ 协议



IP 网络所提供的是不可靠的传输





理想的传输条件特点

- 理想的传输条件：
 - ① 传输信道不产生差错。
 - ② 接收方总是来得及处理收到的数据。
- 理想传输条件下，能够直接实现可靠传输。
- 实际的网络：不具备以上两个理想条件。必须使用一些可靠传输协议，在不可靠的传输信道实现可靠传输。

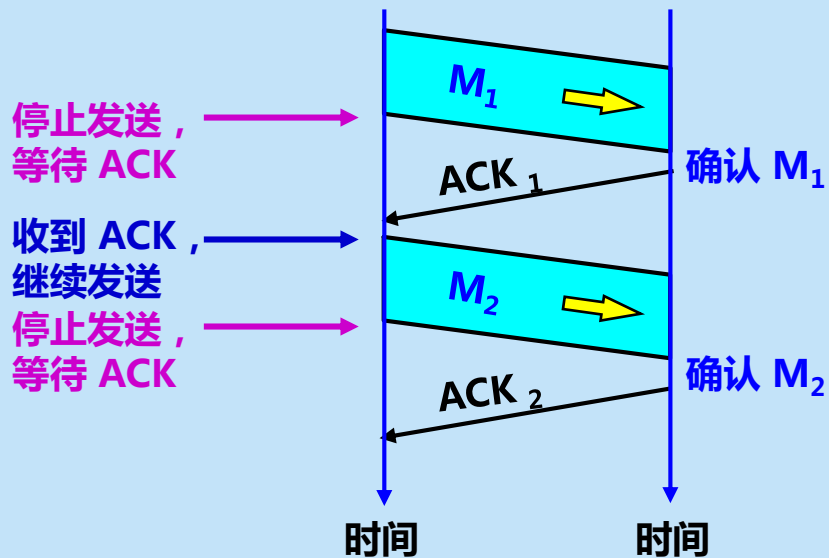


5.4.1 停止等待协议

- “停止等待”就是每发送完一个分组就停止发送，等待对方的确认。在收到确认后再发送下一个分组。
- 全双工通信的双方既是发送方也是接收方。
- 为了讨论问题的方便，我们仅考虑 A 发送数据，而 B 接收数据并发送确认。因此 A 叫做**发送方**，而 B 叫做**接收方**。



1. 无差错情况



A 发送分组 M_1 ，发完就暂停发送，等待 B 的确认 (ACK)。B 收到了 M_1 向 A 发送 ACK。A 在收到了对 M_1 的确认后，就再发送下一个分组 M_2 。



2. 出现差错

- 在接收方 B 会出现两种情况：
 1. B 接收 M_1 时检测出了差错，就**丢弃** M_1 ，其他什么也不做（不通知 A 收到有差错的分组）。
 2. M_1 在传输过程中**丢失了**，这时 B 当然什么都不知道，也什么都不做。
- 在这两种情况下，**B 都不会发送任何信息。**
- 但**A都必须重发分组**，直到B正确接收为止，这样才能实现可靠通信。



2. 出现差错

- **问题**：A如何知道 B 是否正确收到了 M_1 呢？
- **解决方法**：超时重传
 1. A 为每一个已发送的分组都设置了一个**超时计时器**。
 2. A 只要在超时计时器到期之前收到了相应的确认，就撤销该超时计时器，继续发送下一个分组 M_2 。
 3. 若A在超时计时器规定时间内没有收到B的确认，就认为分组错误或丢失，就重发该分组。

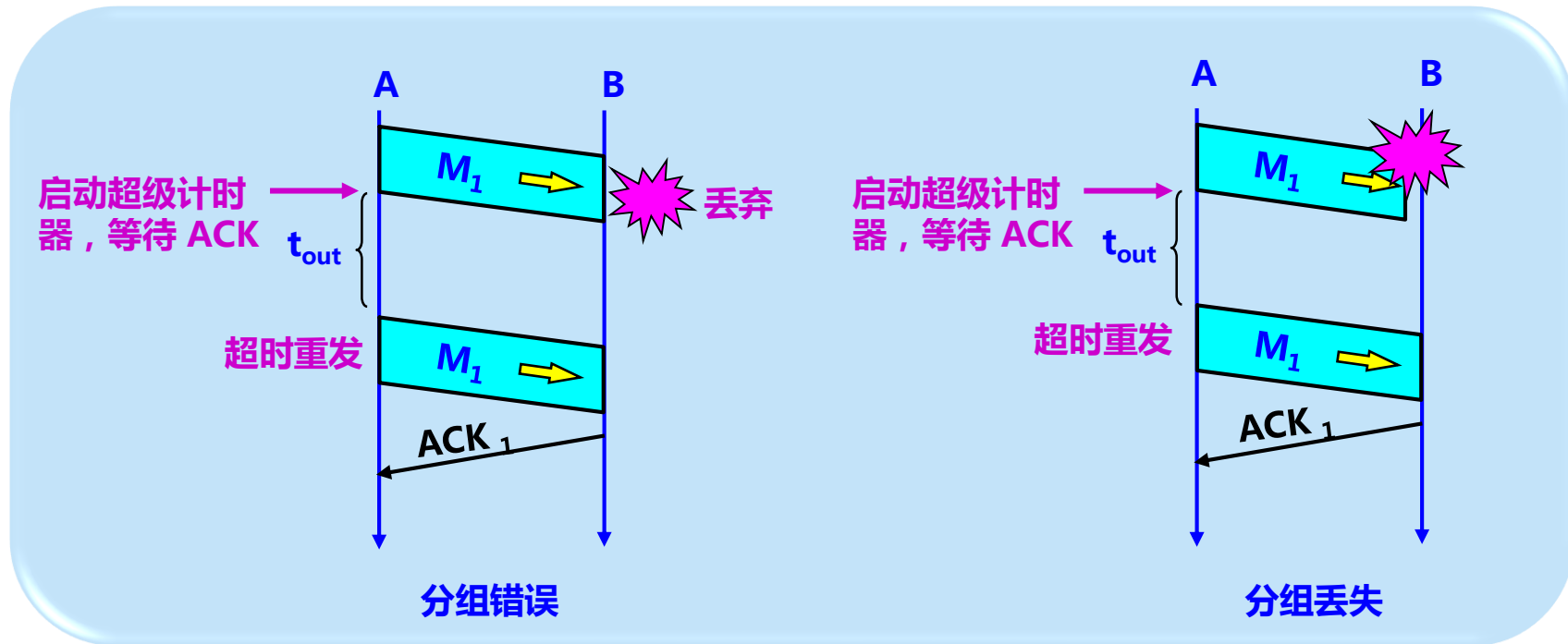


2. 出现差错

- **问题：**若分组正确到达B，但B回送的确认丢失或延迟了，A未收到B的确认，会超时重发。B可能会收到重复的 M_1 。B如何知道收到了重复的分组，需要丢弃呢？
- **解决方法：**编号
- A为每一个发送的分组都进行编号。若B收到了编号相同的分组，则认为收到了重复分组，丢弃重复的分组，并回送确认。
- B为发送的确认也进行编号，指示该确认是对哪一个分组的确认。
- A根据确认及其编号，可以确定它是对哪一个分组的确认，避免重发发送。若为重复的确认，则将其丢弃。



2. 分组出现差错



解决方法：超时重传



3. 确认丢失和确认迟到

● 确认丢失

1. 若 B 所发送的对 M_1 的确认丢失了, 那么 A 在设定的超时重传时间内不能收到确认, 但 A 并无法知道: 是自己发送的分组出错、丢失了, 或者是 B 发送的确认丢失了。因此 A 在超时计时器到期后就要重传 M_1 。
2. 假定 B 又收到了重传的分组 M_1 。这时 B 应采取两个行动:
 - 第一, 丢弃这个重复的分组 M_1 , 不向上层交付。
 - 第二, 向 A 发送确认。不能认为已经发送过确认就不再发送, 因为 A 之所以重传 M_1 就表示 A 没有收到对 M_1 的确认。



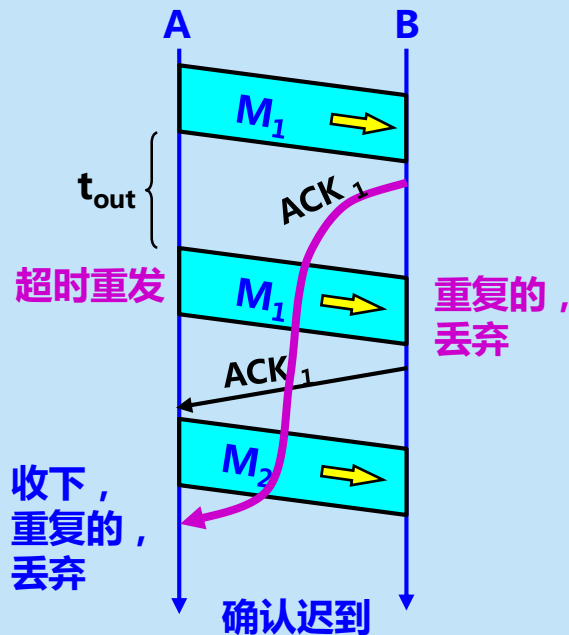
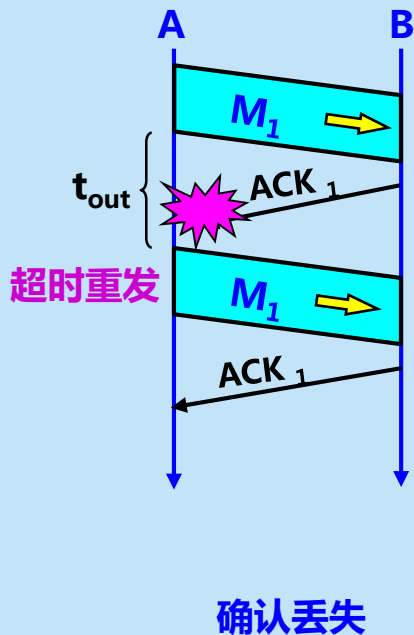
3. 确认丢失和确认迟到

● 确认迟到

1. 传输过程中没有出现差错，但 B 对分组 M_1 的确认迟到了。
2. A 会收到重复的确认。对重复的确认的处理很简单：收下后就丢弃。
3. B 仍然会收到重复的 M_1 ，并且同样要丢弃重复的 M_1 ，并重传确认分组。



3. 确认丢失和确认迟到



重复组分解决方法: 编号



请注意

- 分组必须**暂时保留**已发送的分组的副本，以备重发。
- **分组和确认分组都必须进行编号。**
- 超时计时器的重传时间应当比数据在分组传输的平均往返时间**更长一些**。



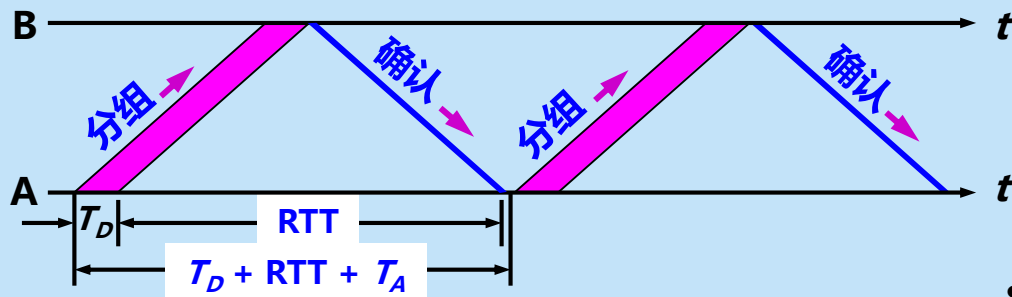
自动重传请求 ARQ

- 通常 A 最终总是可以收到对所有发出的分组的确认。如果 A 不断重传分组但总是收不到确认，就说明通信线路太差，不能进行通信。
- 使用上述的确认和重传机制，我们就可以在不可靠的传输网络上实现可靠的通信。
- 像上述的这种可靠传输协议常称为**自动重传请求 ARQ** (Automatic Repeat reQuest)。意思是重传的请求是自动进行的，接收方不需要请求发送方重传某个出错的分组。



4. 信道利用率

停止等待协议的优点是简单，缺点是信道利用率太低。



停止等待协议的信道利用率太低

- $RTT \gg T_D$
- 重传影响

信道利用率
$$U = \frac{T_D}{T_D + RTT + T_A} \quad (5-3)$$



停止等待协议要点

- **停止等待**。发送方每次只发送一个分组。在收到确认后再发送下一个分组。
- **编号**。对发送的每个分组和确认都进行编号。
- **自动重传请求**。发送方为每个发送的分组设置一个超时计时器。若超时计时器超时，发送方会自动重传分组。
- 简单，但信道利用率太低。



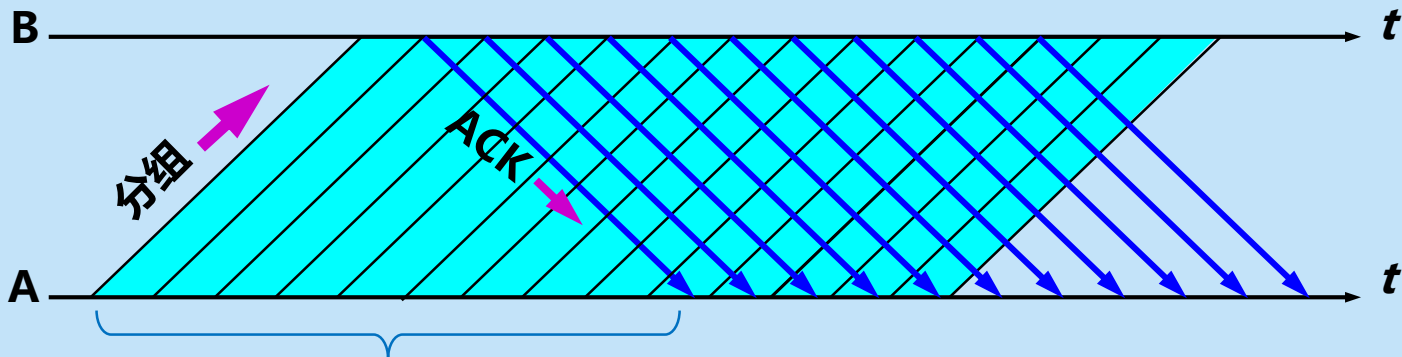
流水线传输

- 为了提高传输效率，发送方可以不使用低效率的停止等待协议，而是采用流水线传输。
- **流水线传输**就是发送方可连续发送多个分组，不必每发完一个分组就停顿下来等待对方的确认。这样可使信道上一一直有数据不间断地传送。
- 由于信道上一一直有数据不间断地传送，这种传输方式可获得很高的信道利用率。



流水线传输

由于信道上一直有数据不间断地传送，
这种传输方式可获得很高的信道利用率。



在收到确认之前，发送方连续发出多个分组

流水线传输可提高信道利用率



5.4.2 连续 ARQ 协议

- **基本思想：**
- 发送方一次可以发出**多个分组**。
- 使用**滑动窗口协议**控制发送方和接收方所能发送和接收的分组的数量和编号。
- 每收到一个确认，发送方就把发送窗口**向前滑动**。
- 接收方一般采用**累积确认**的方式。
- 采用**回退N** (Go-Back-N) 方法进行重传。



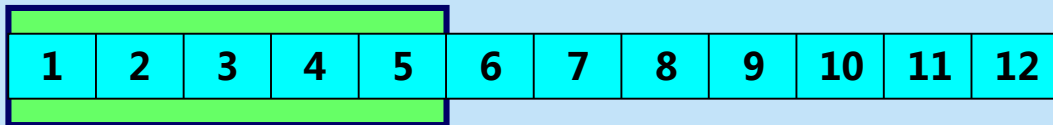
5.4.2 连续 ARQ 协议

- 滑动窗口协议比较复杂，是 TCP 协议的精髓所在。
- 发送方维持的**发送窗口**，它的意义是：**位于发送窗口内的分组都可连续发送出去，而不需要等待对方的确认。这样，信道利用率就提高了。**
- 连续 ARQ 协议规定，**发送方每收到一个确认，就把发送窗口向前滑动一个分组的位置。**



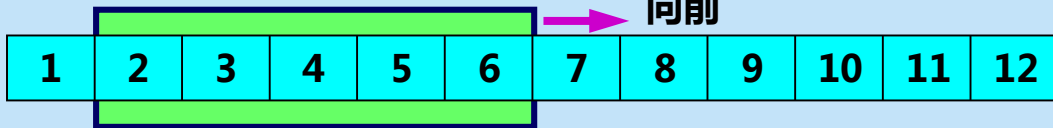
5.4.2 连续 ARQ 协议

发送窗口



(a) 发送方维持发送窗口 (发送窗口是 5)

发送窗口



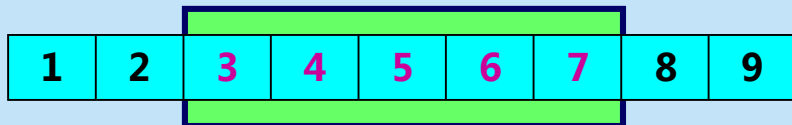
(b) 收到一个确认后发送窗口向前滑动

连续 ARQ 协议的工作原理



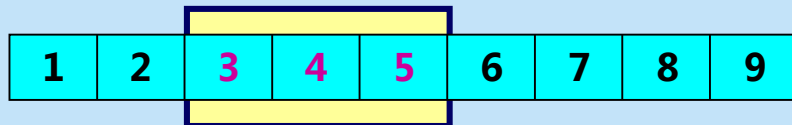
滑动窗口协议

发送窗口



(a) 发送方维持发送窗口

接收窗口

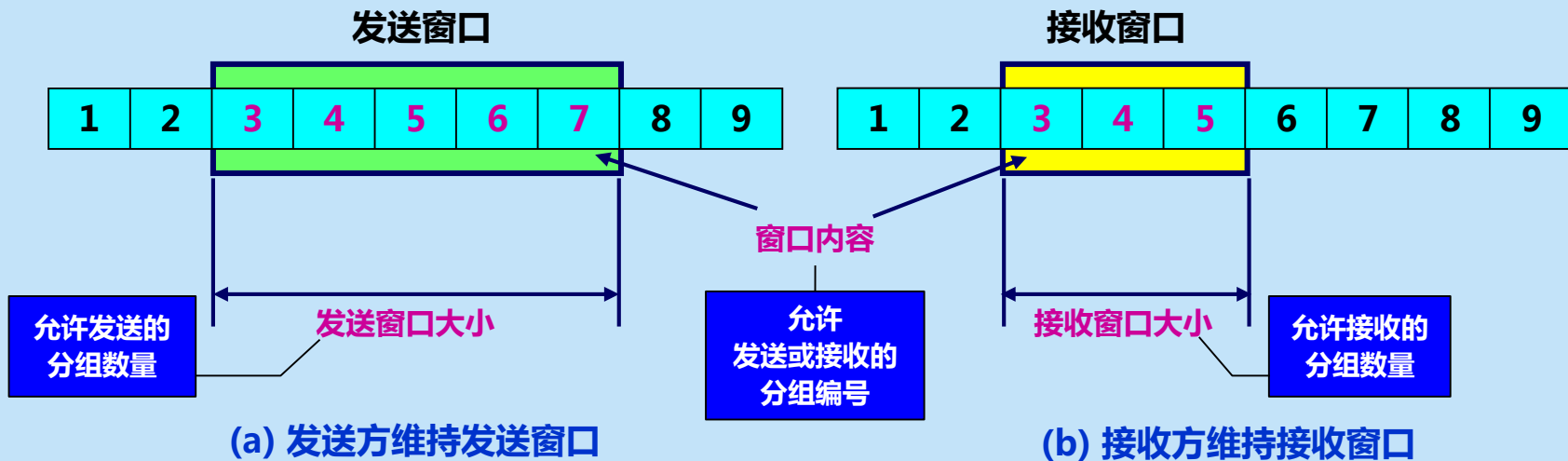


(b) 接收方维持接收窗口

发送方和接收方分别维持发送窗口和接收窗口



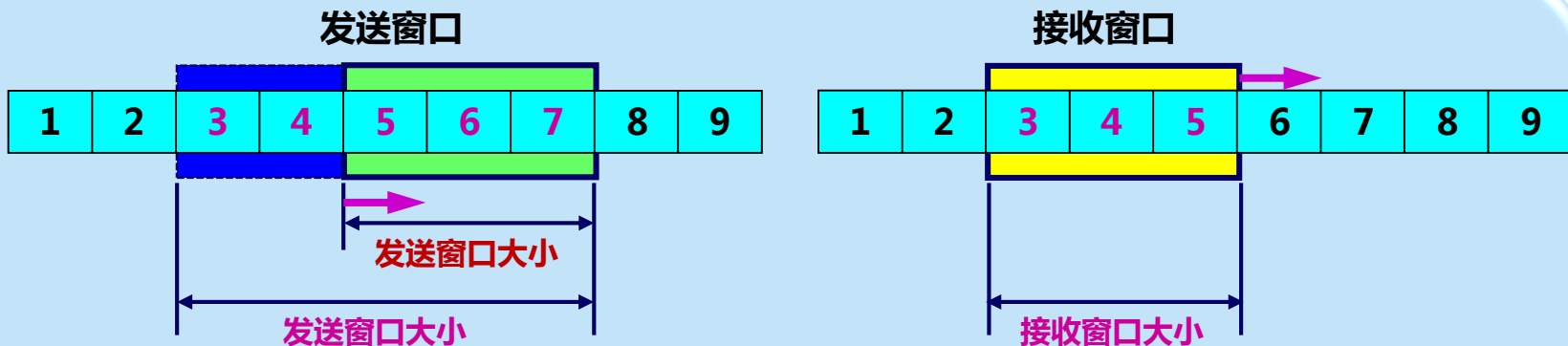
滑动窗口协议



发送方和接收方分别维持发送窗口和接收窗口



滑动窗口协议

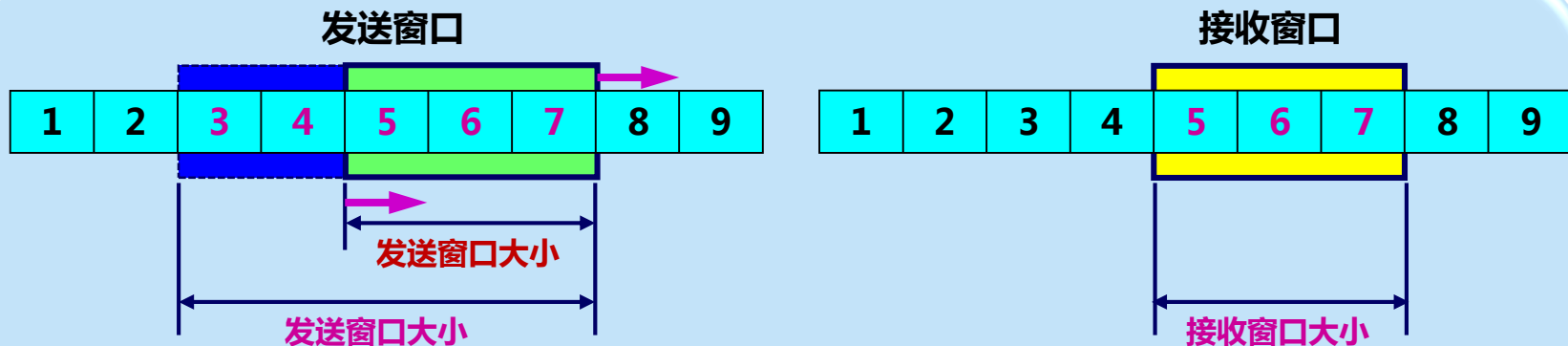


发送后，在收到确认前，发送窗口会变小

发送方和接收方分别维持发送窗口和接收窗口



滑动窗口协议

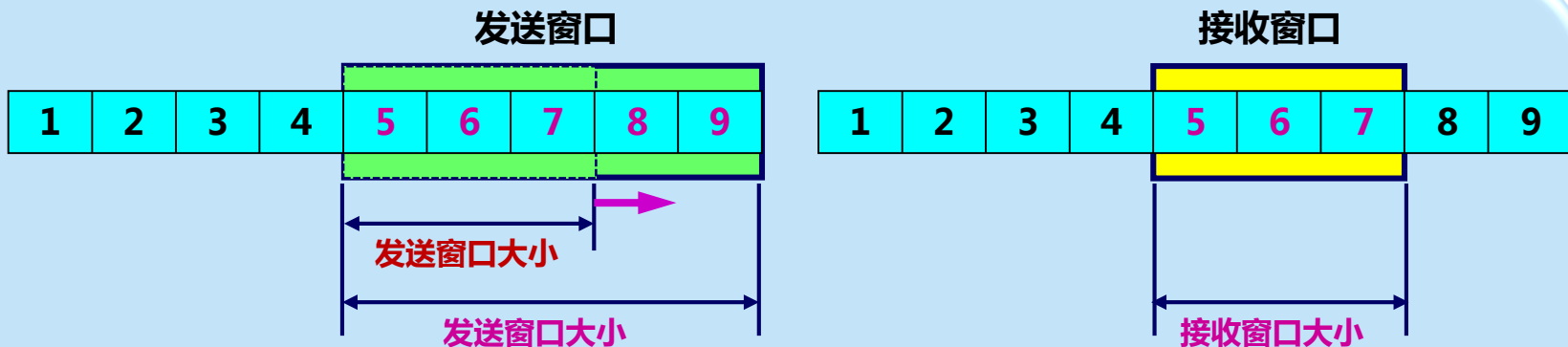


接收的分组正确，向前滑动接收窗口

发送方和接收方分别维持发送窗口和接收窗口



滑动窗口协议



收到确认后，向前滑动发送窗口，窗口变大

发送方和接收方分别维持发送窗口和接收窗口

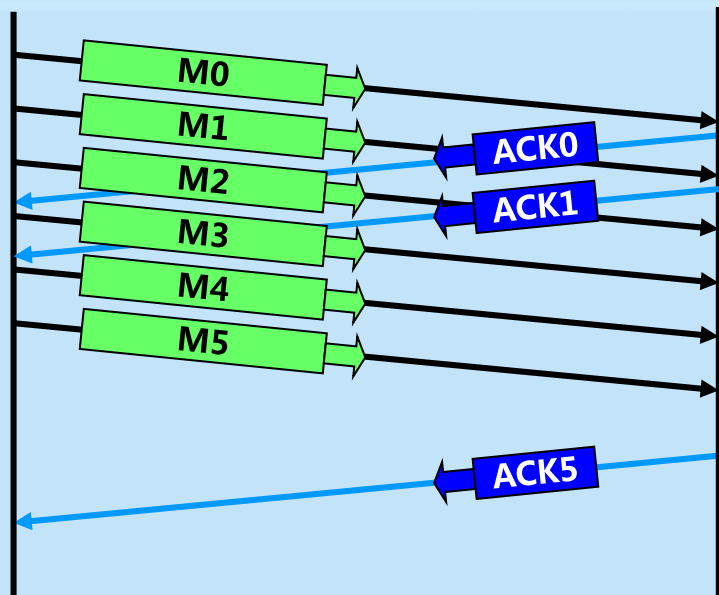


累积确认

- 接收方一般采用**累积确认**的方式。即不必对收到的分组逐个发送确认，而是**对按序到达的最后一个分组发送确认**，这样就表示：**到这个分组为止的所有分组都已正确收到了**。
- **优点**：容易实现，即使确认丢失也不必重传。
- **缺点**：不能向发送方反映出接收方已经正确收到的所有分组的信息。



累积确认



ACK0 确认 M0，将M0提交给上层协议或用户

ACK1 确认 M1，将M1提交给上层协议或用户

M2 正确

M3 正确

M4 正确

M5 正确

ACK5 为累积确认，
表示 M5 及之前的
M2、3、4 都正确。

将M2、M3、M4、M5
提交给上层协议或用户

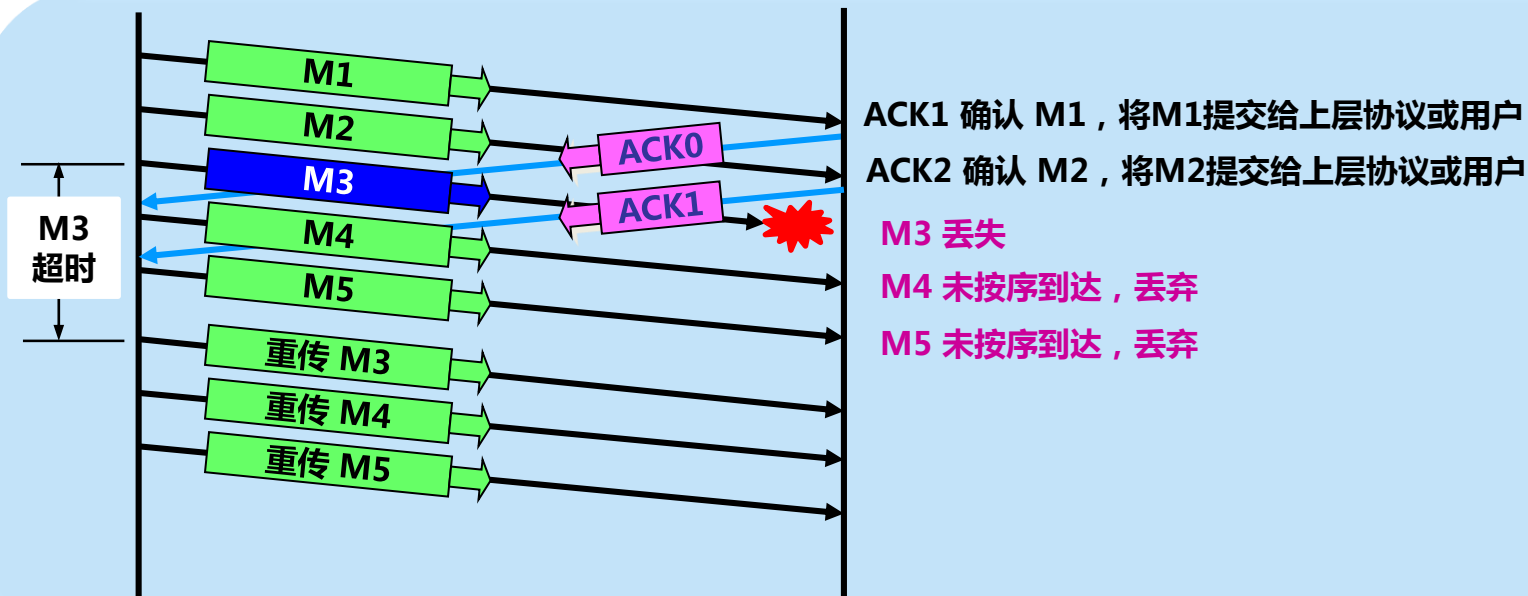


Go-back-N (回退 N)

- 如果发送方发送了前 5 个分组，而中间的第 3 个分组丢失了。这时接收方只能对前两个分组发出确认。发送方无法知道后面三个分组的下落，而只好把后面的三个分组都再重传一次。
- 这就叫做 Go-back-N (回退 N)，表示需要再退回来重传已发送过的 N 个分组。
- 可见当通信线路质量不好时，连续 ARQ 协议会带来负面的影响。



Go-back-N (回退 N)



连续 ARQ 协议采用回退N (Go-Back-N) 重传



TCP 可靠通信的具体实现

- TCP 连接的每一端都必须设有两个窗口——一个**发送窗口**和一个**接收窗口**。
- TCP 的可靠传输机制用**字节的序号**进行控制。TCP 所有的确认都是基于序号而不是基于报文段。
- TCP 两端的四个窗口经常处于**动态变化**之中。
- TCP连接的往返时间 RTT 也不是固定不变的。需要使用特定的算法**估算较为合理的重传时间**。



连续 ARQ 协议与停止等待协议

	连续ARQ协议	停止等待协议
发送的分组数量	一次发送多个分组	一次发送一个分组
传输控制	滑动窗口协议	停止-等待
确认	单独确认 + 累积确认	单独确认
超时定时器	每个发送的分组	
编号	每个发送的分组	
重传	回退N, 多个分组	一个分组



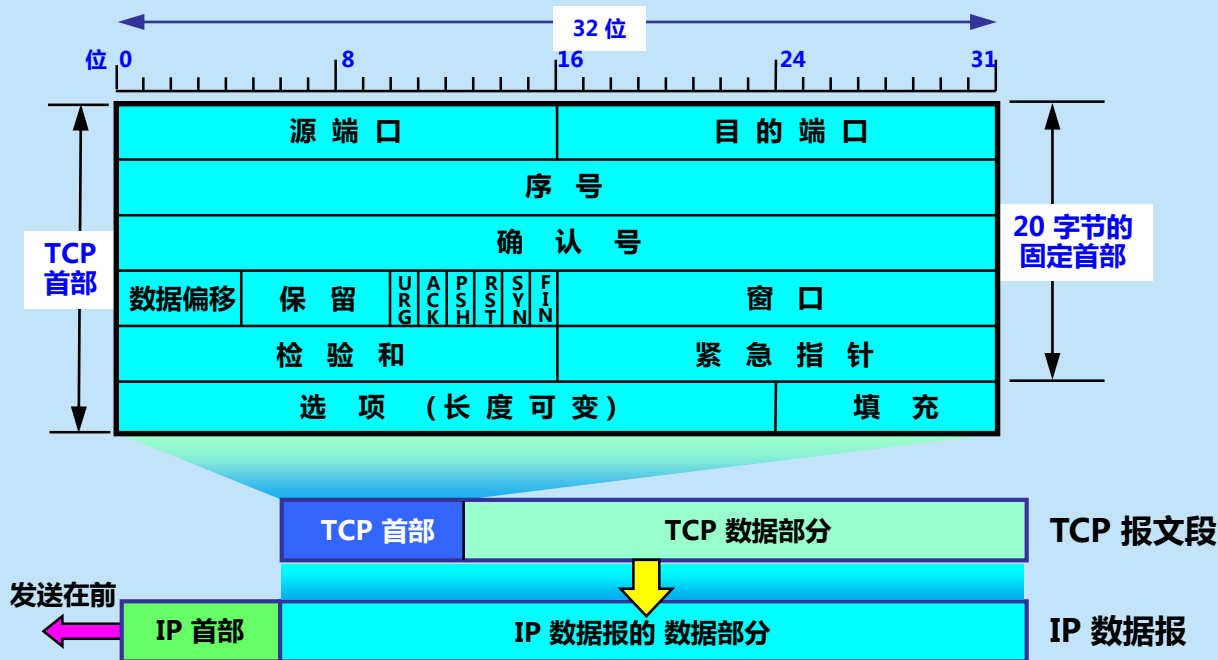
5.5 TCP 报文段的首部格式

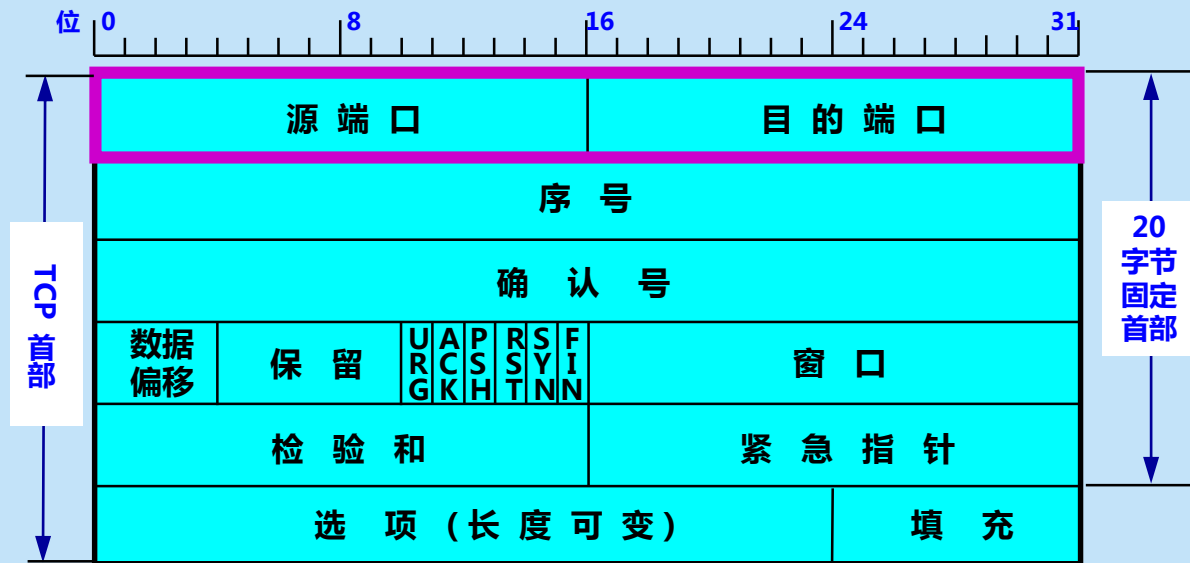
- TCP 虽然是面向字节流的，但 TCP 传送的数据单元却是报文段。
- 一个 TCP 报文段分为首部和数据两部分，而 TCP 的全部功能都体现在它首部中各字段的作用。
- TCP 报文段首部的 前 20 个字节是固定的，后面有 $4n$ 字节是根据需要而增加的选项（ n 是整数）。因此 TCP 首部的最小长度是 20 字节。



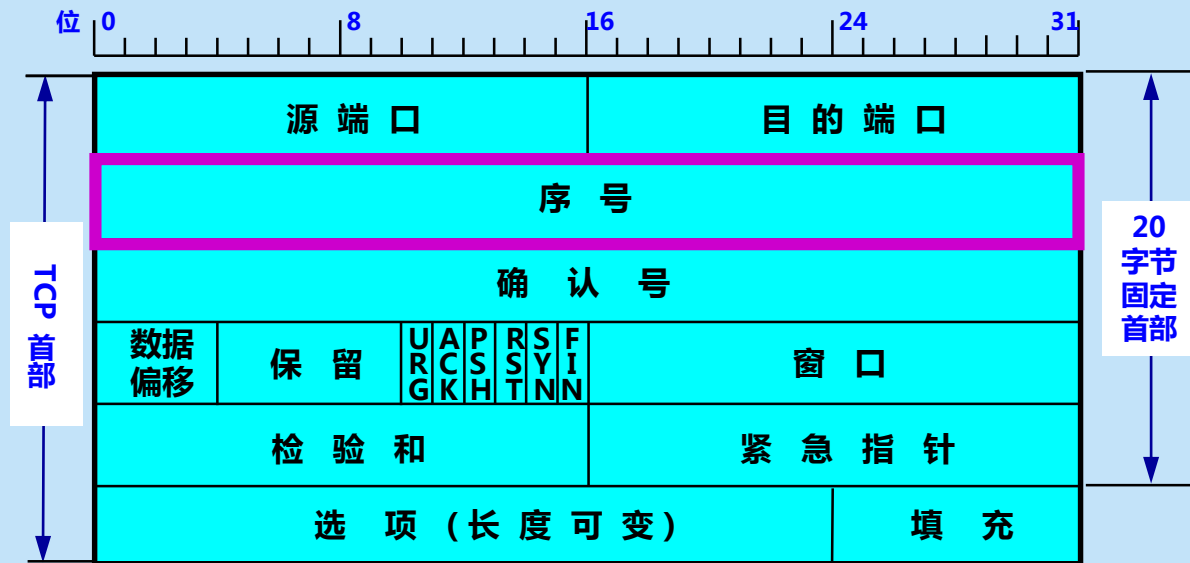
5.5 TCP 报文段的首部格式

**TCP首部的
最小长度是
20字节。**

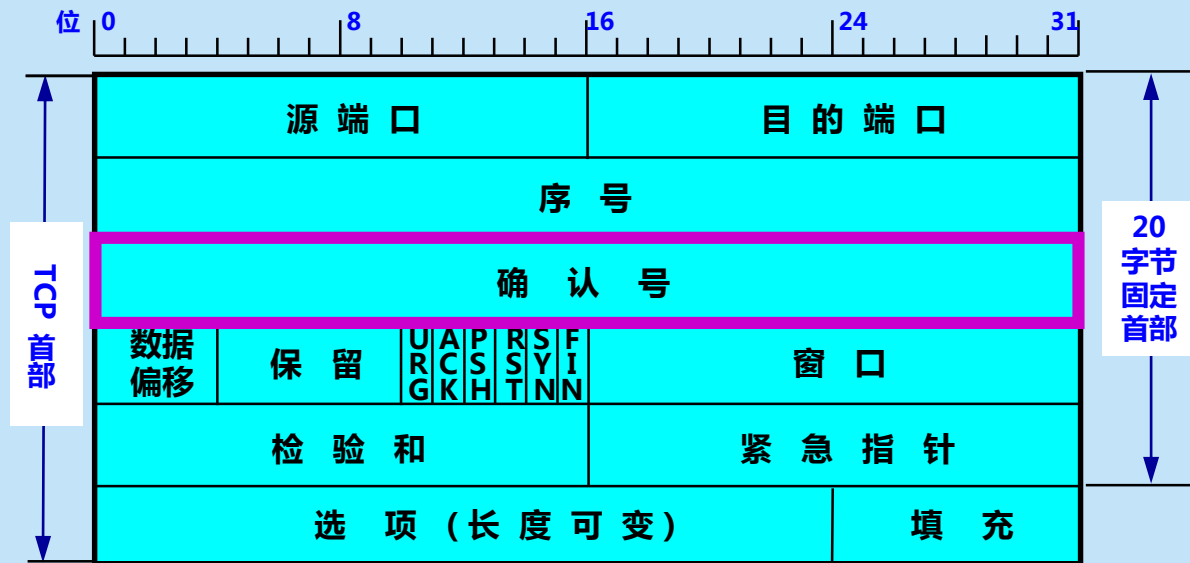




源端口和目的端口字段——各占 2 字节。端口是运输层与应用层的服务接口。运输层的复用和分用功能都要通过端口才能实现。



序号字段——占 4 字节。TCP 连接中传送的数据流中的每一个字节都编上一个序号。序号字段的值则指的是本报文段所发送的数据的第一个字节的序号。



确认号字段——占 4 字节，是期望收到对方的下一个报文段的数据的第一个字节的序号。

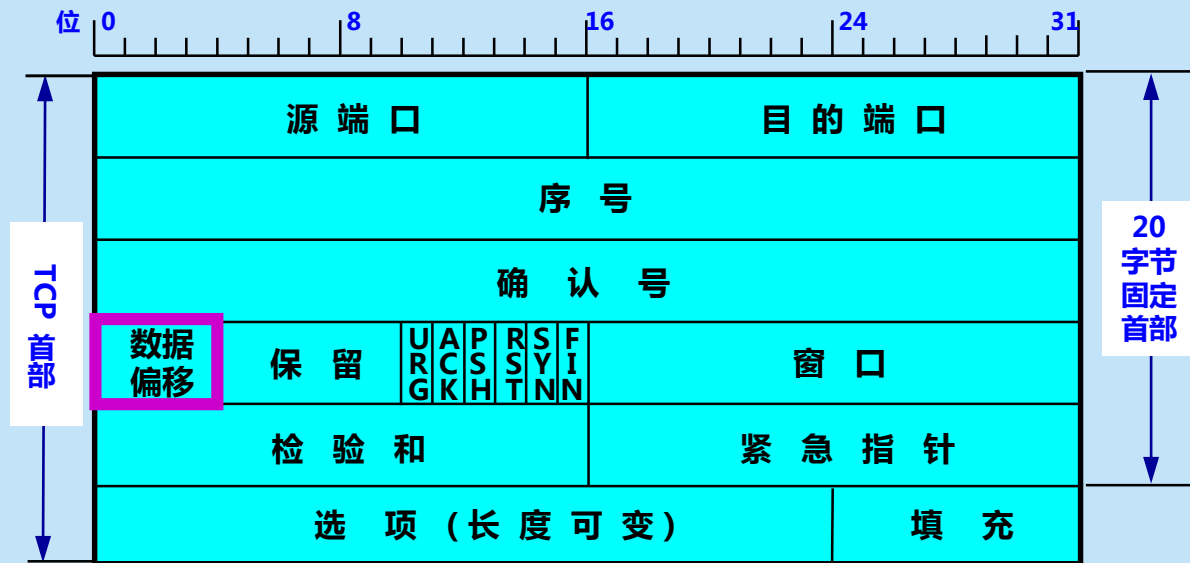


现有1000个字节的数据，字节编号为1000-1999。

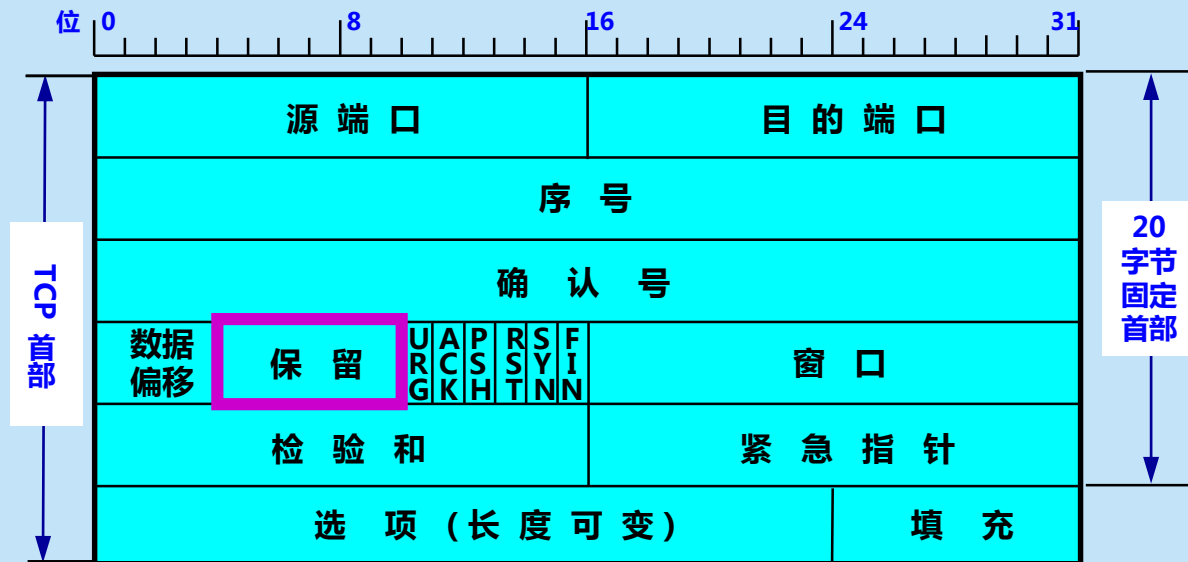
1000	1001	1002	1003				1997	1998	1999
------	------	------	------	-------	-------	-------	------	------	------

报文段的序号？

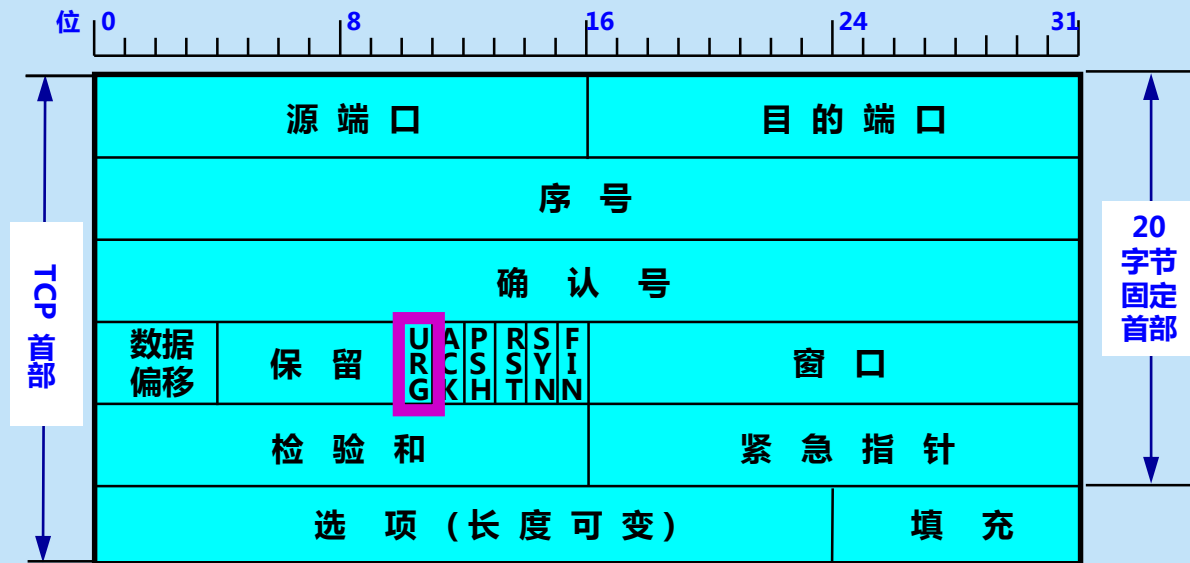
确认号？



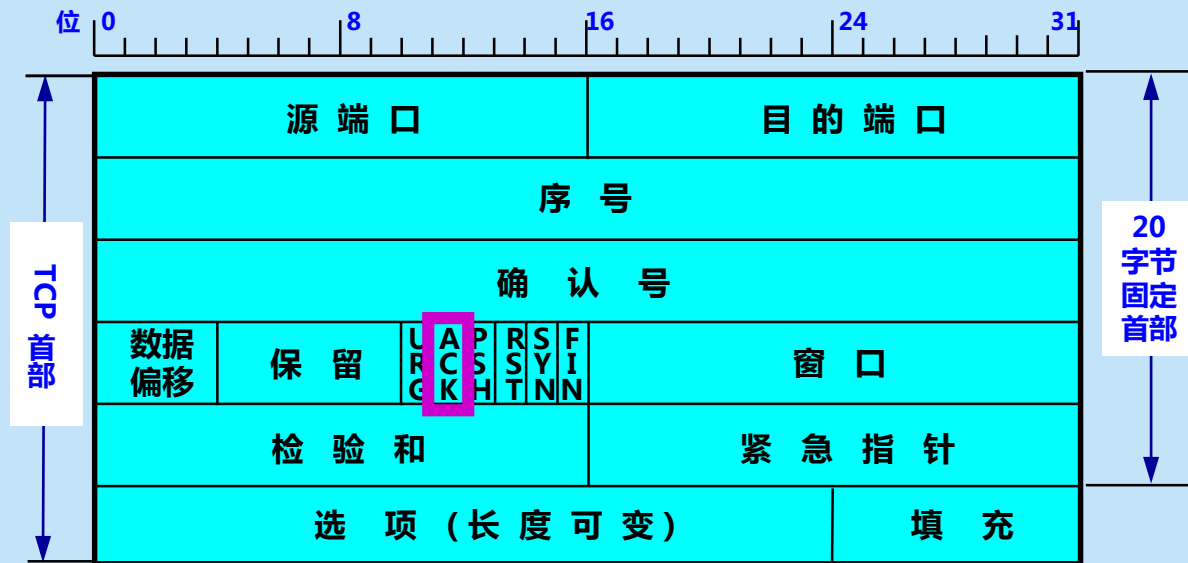
数据偏移 (即首部长度的) —— 占 4 位，它指出 TCP 报文段的数据起始处距离 TCP 报文段的起始处有多远。“数据偏移”的单位是 32 位字 (以 4 字节为计算单位)。



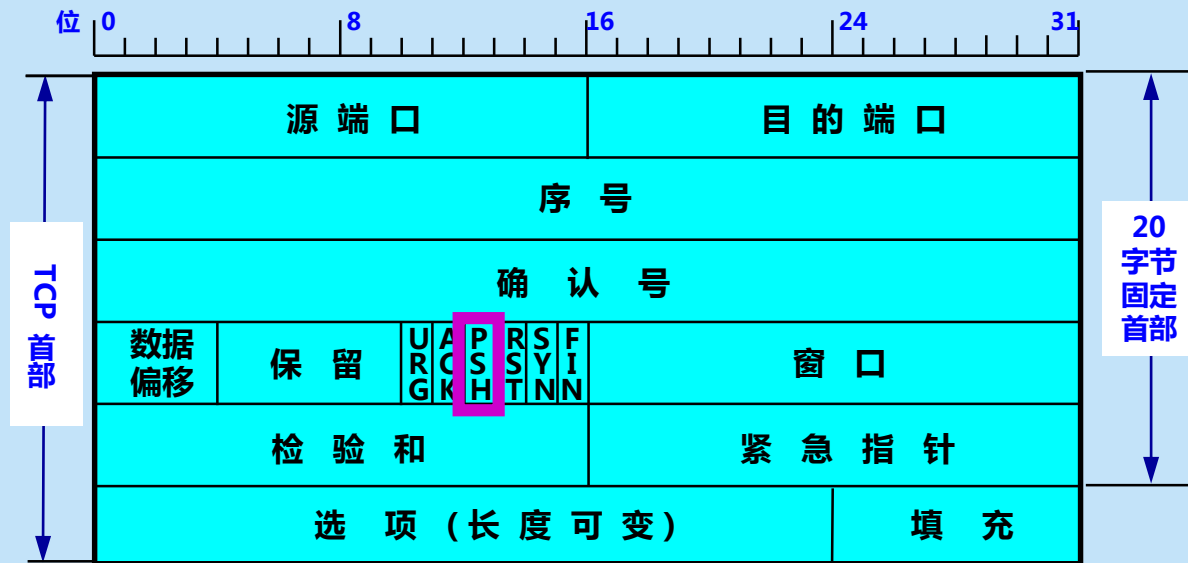
保留字段——占 6 位，保留为今后使用，但目前应置为 0。



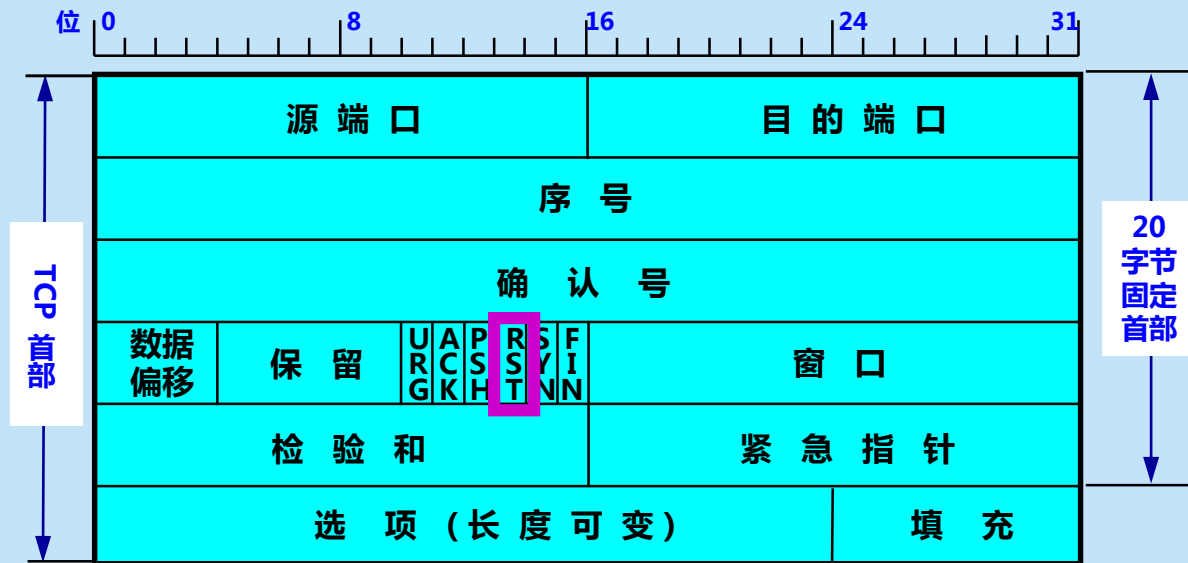
紧急 URG —— 当 URG = 1 时，表明紧急指针字段有效。它告诉系统此报文段中有紧急数据，应尽快传送(相当于高优先级的数据)。



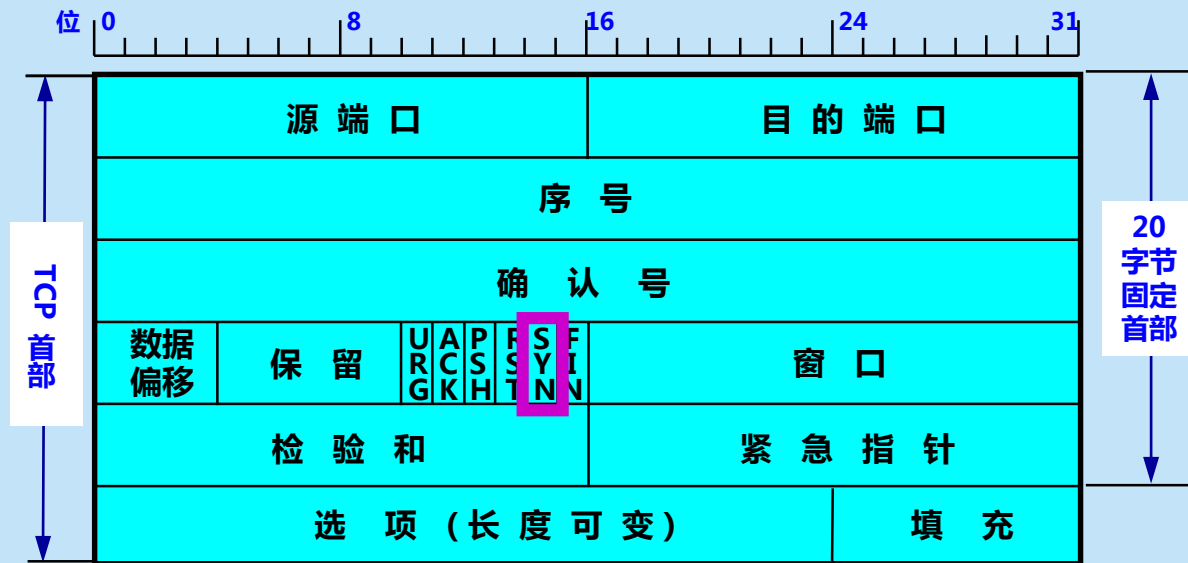
确认 ACK —— 只有当 ACK = 1 时确认号字段才有效。当 ACK = 0 时，确认号无效。



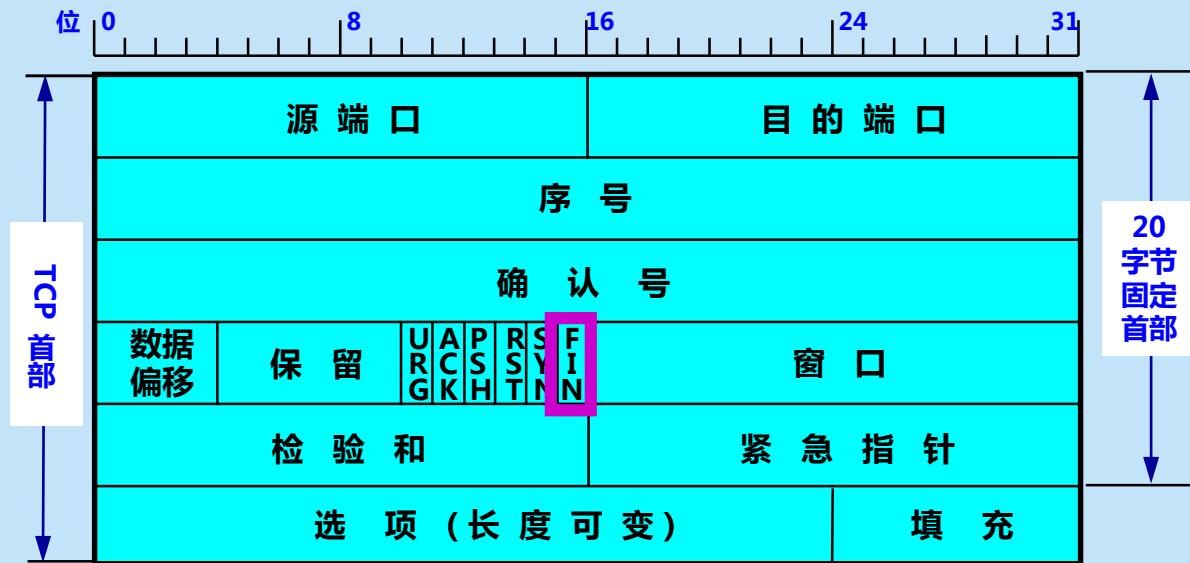
推送 PSH (PuSH) —— 接收 TCP 收到 PSH = 1 的报文段，就尽快地交付接收应用进程，而不再等到整个缓存都填满了后再向上交付。



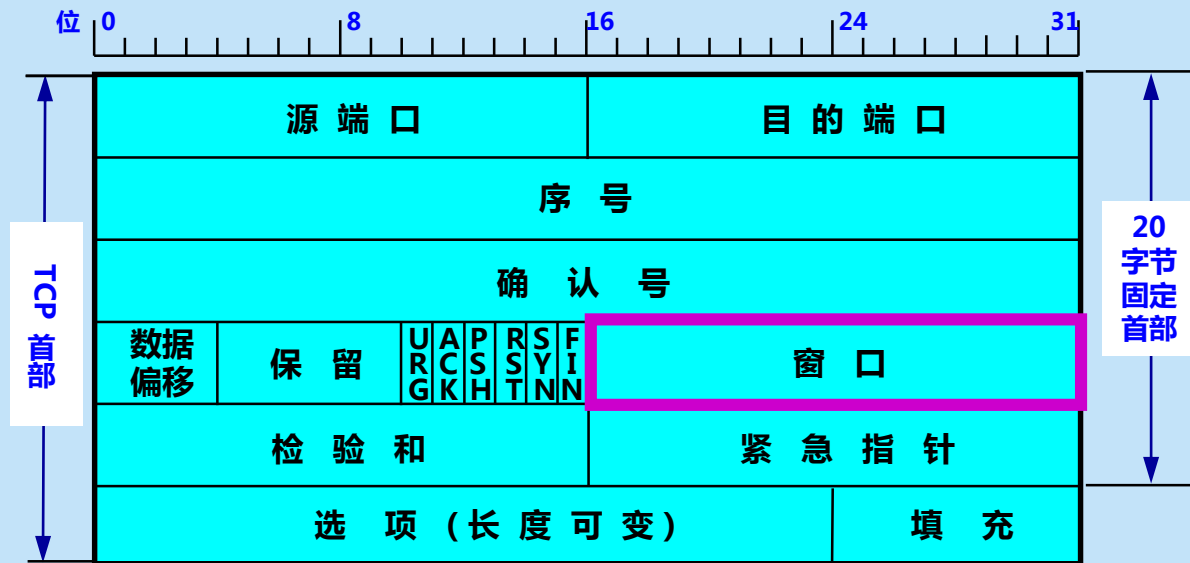
复位 RST (ReSeT) —— 当 RST=1 时，表明 TCP 连接中出现严重差错（如由于主机崩溃或其他原因），必须释放连接，然后再重新建立运输连接。



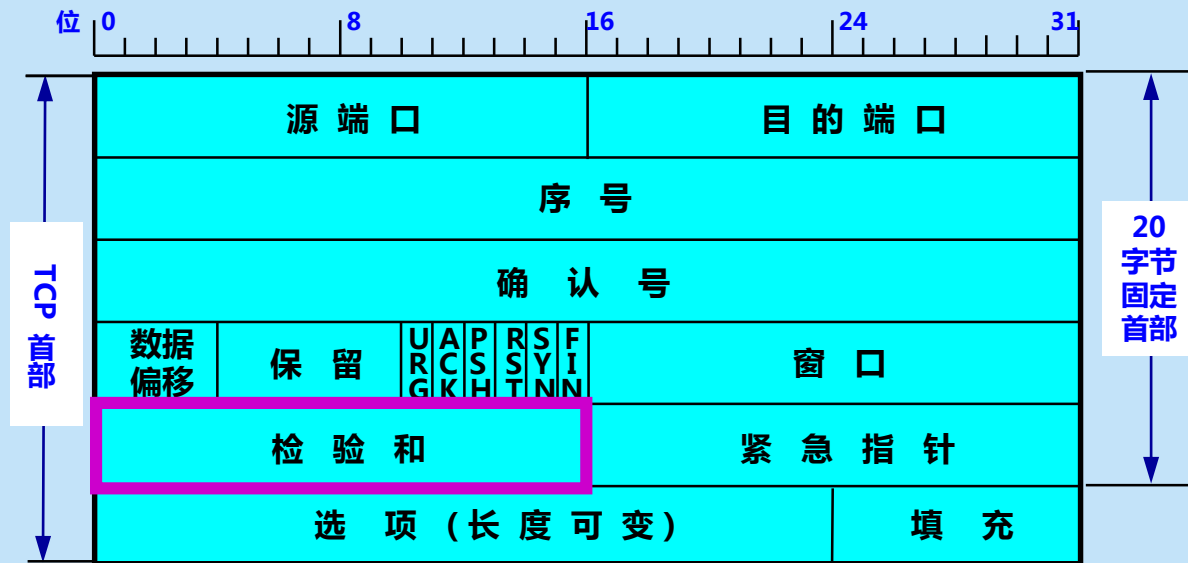
同步 SYN —— 同步 SYN = 1 表示这是一个连接请求或连接接受报文。



终止 FIN (FINish) —— 用来释放一个连接。FIN=1 表明此报文段的发送端的数据已发送完毕，并要求释放运输连接。



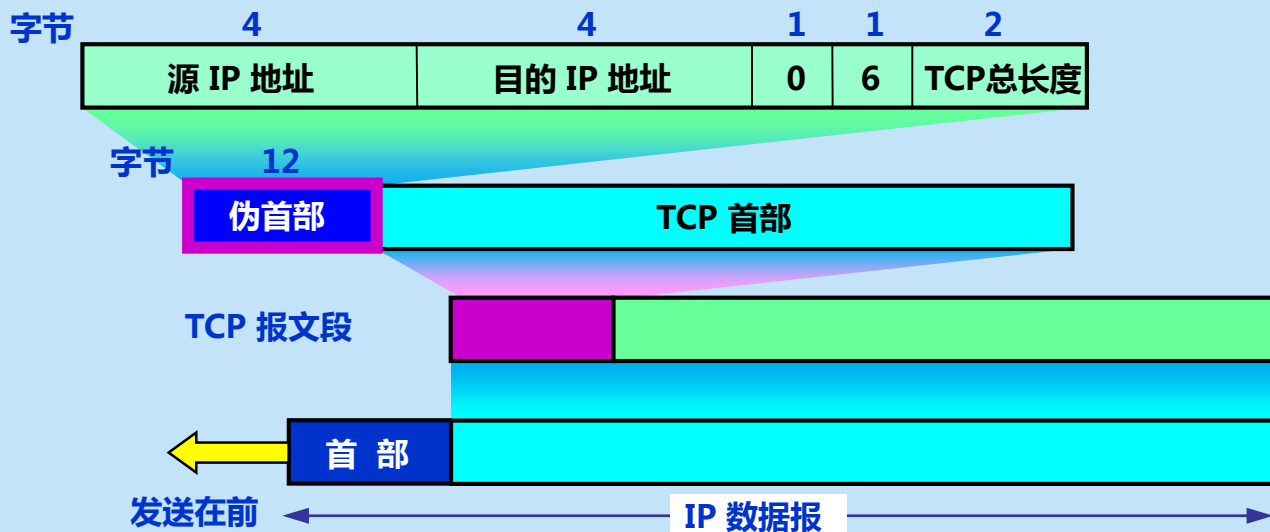
窗口字段 —— 占 2 字节，用来让对方设置发送窗口的依据，单位为字节。

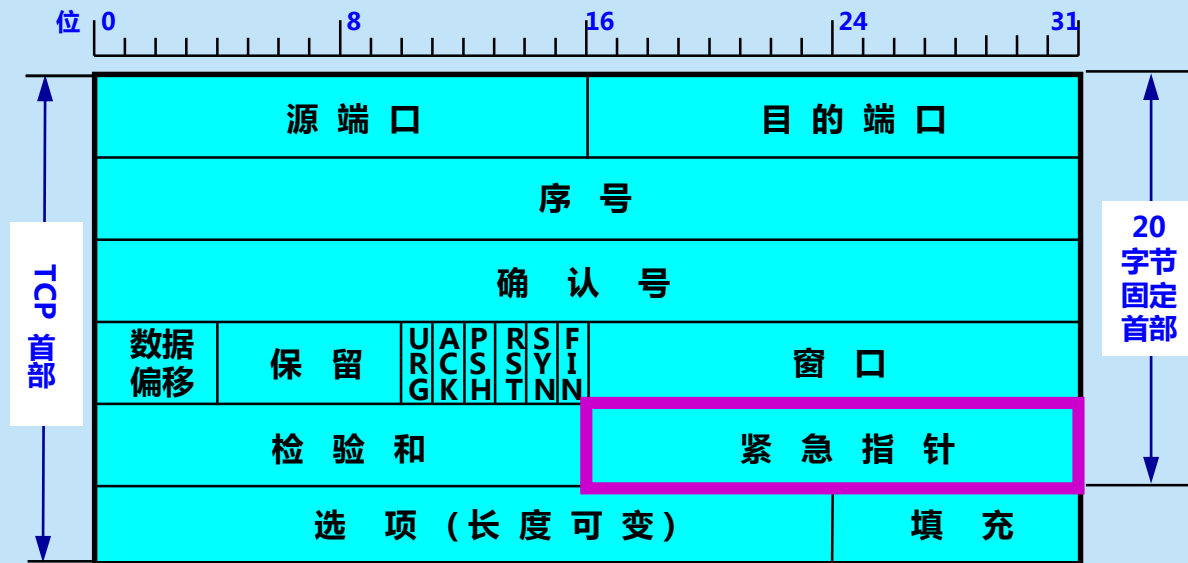


**检验和 —— 占 2 字节。检验和字段检验的范围包括首部和数据这两部分。
在计算检验和时，要在 TCP 报文段的前面加上 12 字节的伪首部。**



在计算检验和时，临时把 12 字节的“伪首部”和 TCP 报文段连接在一起。伪首部仅仅是为了计算检验和。





紧急指针字段 —— 占 16 位，指出在本报文段中紧急数据共有多少个字节（紧急数据放在本报文段数据的最前面）。



MSS (Maximum Segment Size)

是 TCP 报文段中的数据字段的最大长度。

数据字段加上 TCP 首部才等于整个的 TCP 报文段。

所以，MSS是“TCP 报文段长度减去 TCP 首部长度的”。



选项字段 —— 长度可变。TCP 最初只规定了一种选项，即最大报文段长度 MSS。MSS 告诉对方 TCP：“我的缓存所能接收的报文段的数据字段的最大长度是 MSS 个字节。”

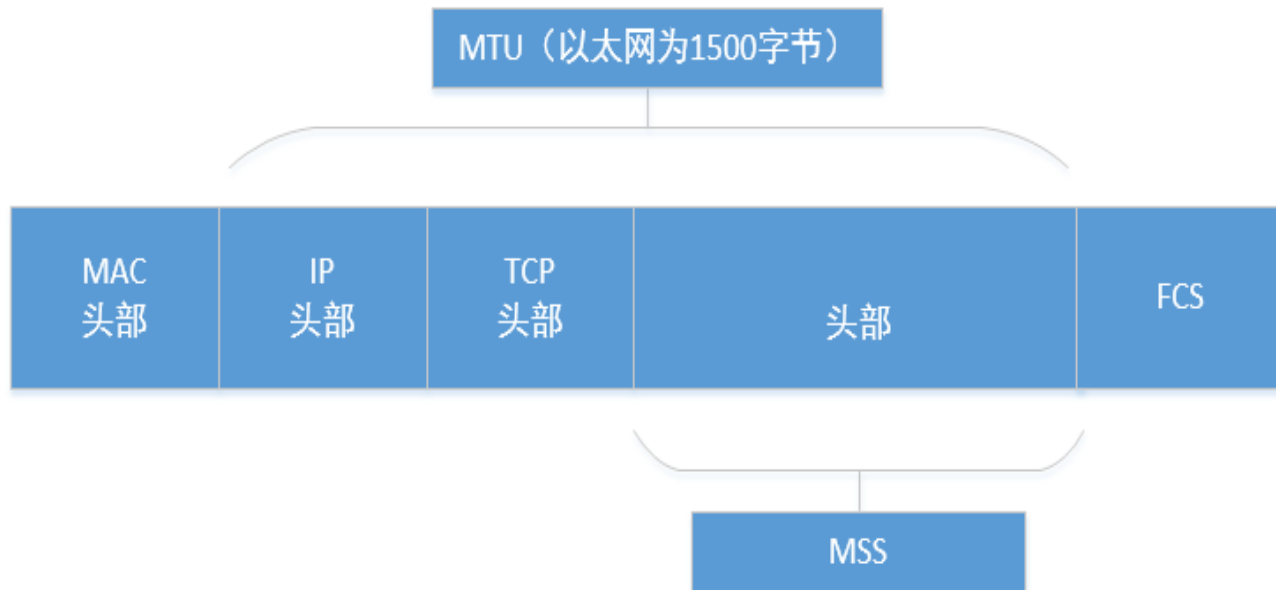


为什么要规定 MSS ？

- **MSS 与接收窗口值没有关系。**
- **若选择较小的 MSS 长度，网络的利用率就降低。**
- **若 TCP 报文段非常长，那么在 IP 层传输时就有可能要分解成多个短数据报片。在终点要把收到的各个短数据报片装配成原来的 TCP 报文段。当传输出错时还要进行重传。这些也都会使开销增大。**
- **因此，MSS 应尽可能大些，只要在 IP 层传输时不需要再分片就行。**
- **但最佳的 MSS 是很难确定的。**



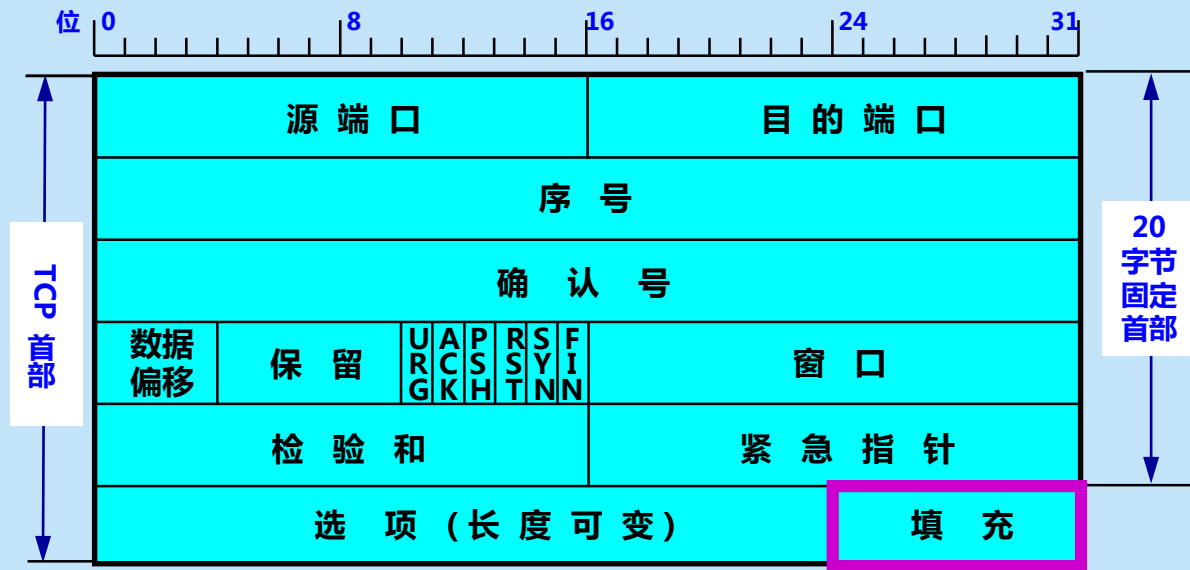
MSS 与 MTU ?





其他选项

- **窗口扩大选项** ——占 3 字节，其中有一个字节表示移位值 S 。新的窗口值等于 TCP 首部中的窗口位数增大到 $(16 + S)$ ，相当于把窗口值向左移动 S 位后获得实际的窗口大小。
- **时间戳选项** ——占 10 字节，其中最主要的字段时间戳值字段（4 字节）和时间戳回送回答字段（4 字节）。
- **选择确认选项** ——在后面的 5.6.3 节介绍。



填充字段 —— 这是为了使整个首部长度的 4 字节的整数倍。



5.6 TCP 可靠传 输的实现

5.6.1

以字节为单位的滑动窗口

5.6.2

超时重传时间的选择

5.6.3

选择确认 SACK

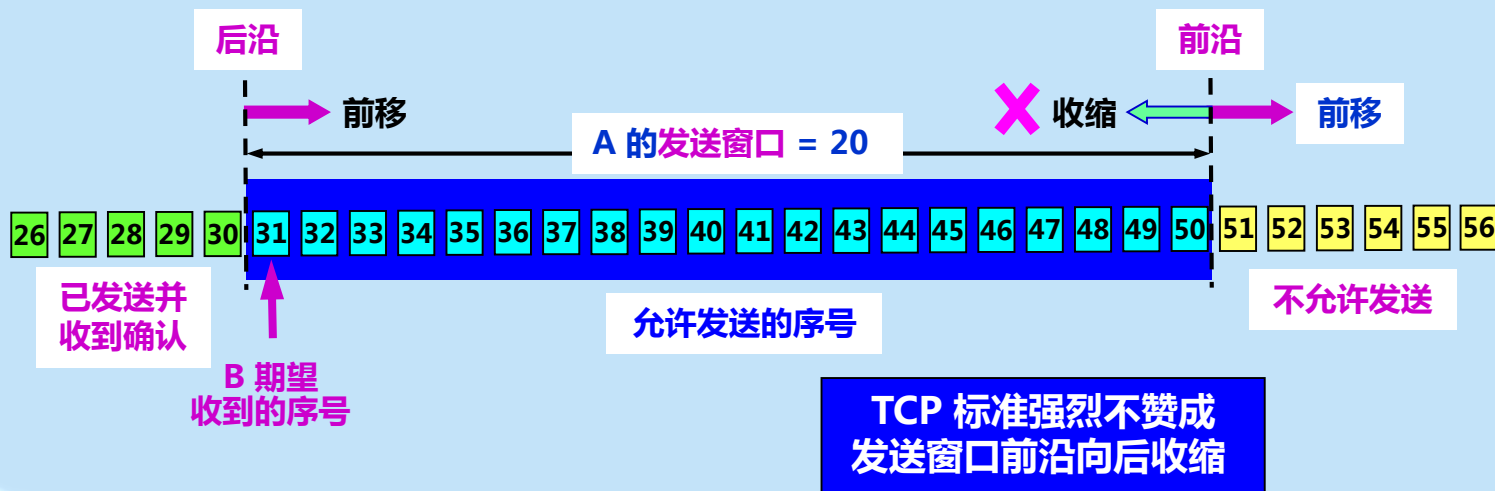


5.6.1 以字节为单位的滑动窗口

- TCP 使用流水线传输和滑动窗口协议实现高效、可靠的传输。
- TCP 的滑动窗口是**以字节为单位的**。
- 发送方 A 和接收方 B 分别维持一个发送窗口和一个接收窗口。
- **发送窗口表示**：在没有收到确认的情况下，可以连续把窗口内的数据全部发送出去。
- **接收窗口表示**：只允许接收落入窗口内的数据。

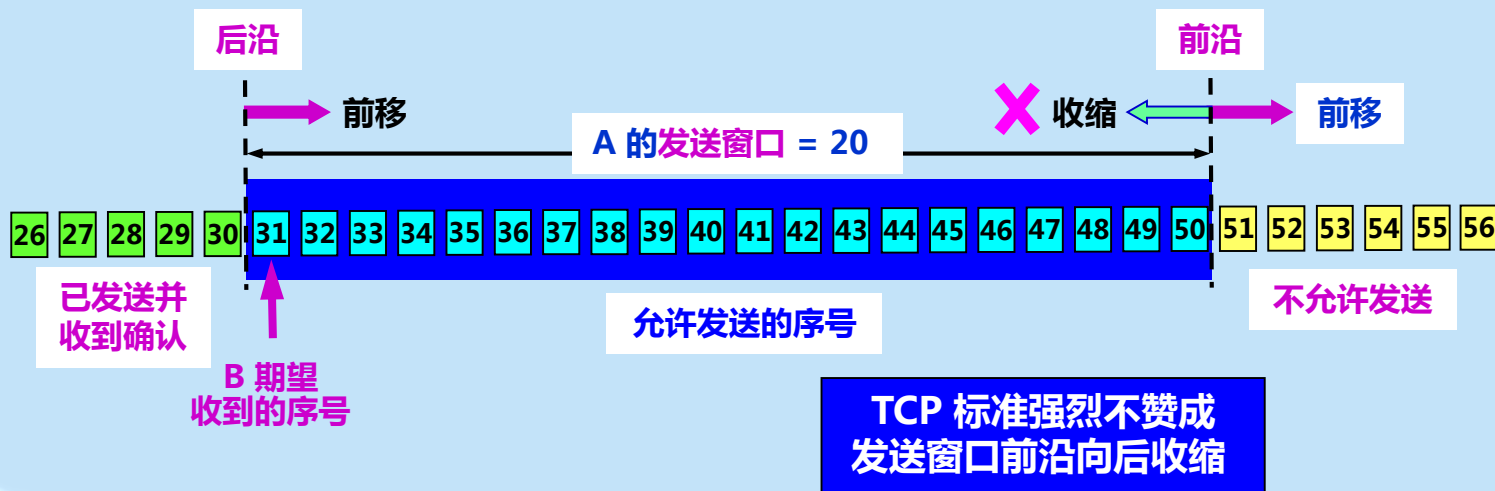


- 根据 B 给出的窗口值，A 构造出自己的发送窗口。
- **发送窗口表示**：在没有收到 B 的确认的情况下，A 可以连续把窗口内的数据都发送出去。
- 发送窗口里面的序号表示允许发送的序号。
- 显然，窗口越大，发送方就可以在收到对方确认之前连续发送更多的数据，因而可能获得更高的传输效率。



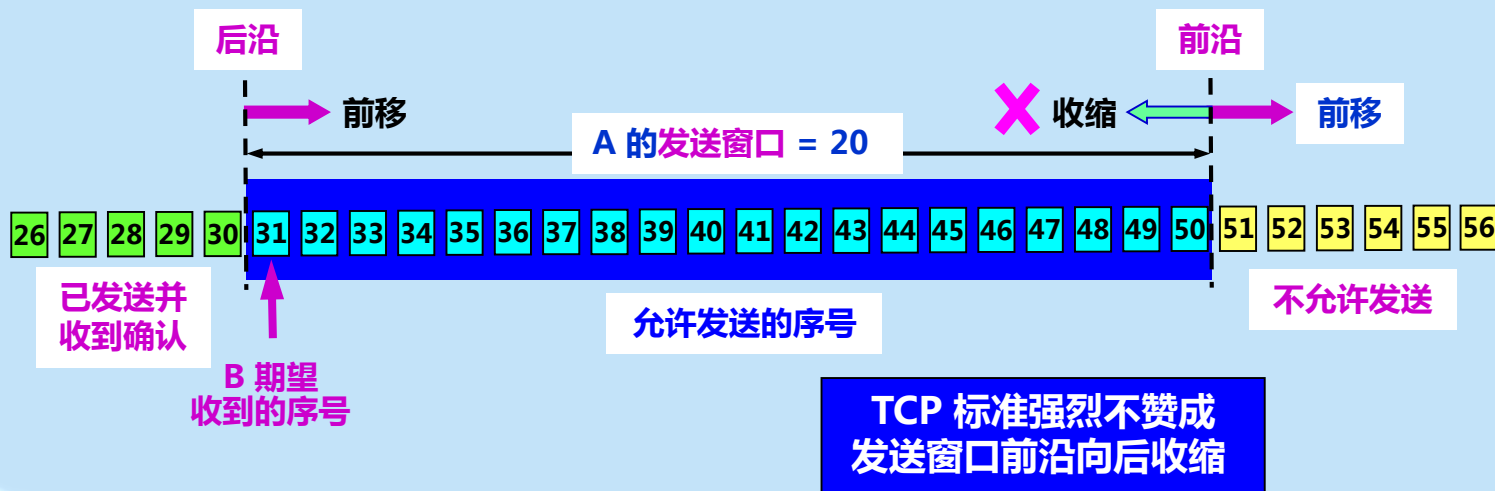


- 根据 B 给出的窗口值，A 构造出自己的发送窗口。
- **发送窗口表示**：在没有收到 B 的确认的情况下，A 可以连续把窗口内的数据都发送出去。
- 发送窗口里面的序号表示允许发送的序号。
- 显然，窗口越大，发送方就可以在收到对方确认之前连续发送更多的数据，因而可能获得更高的传输效率。



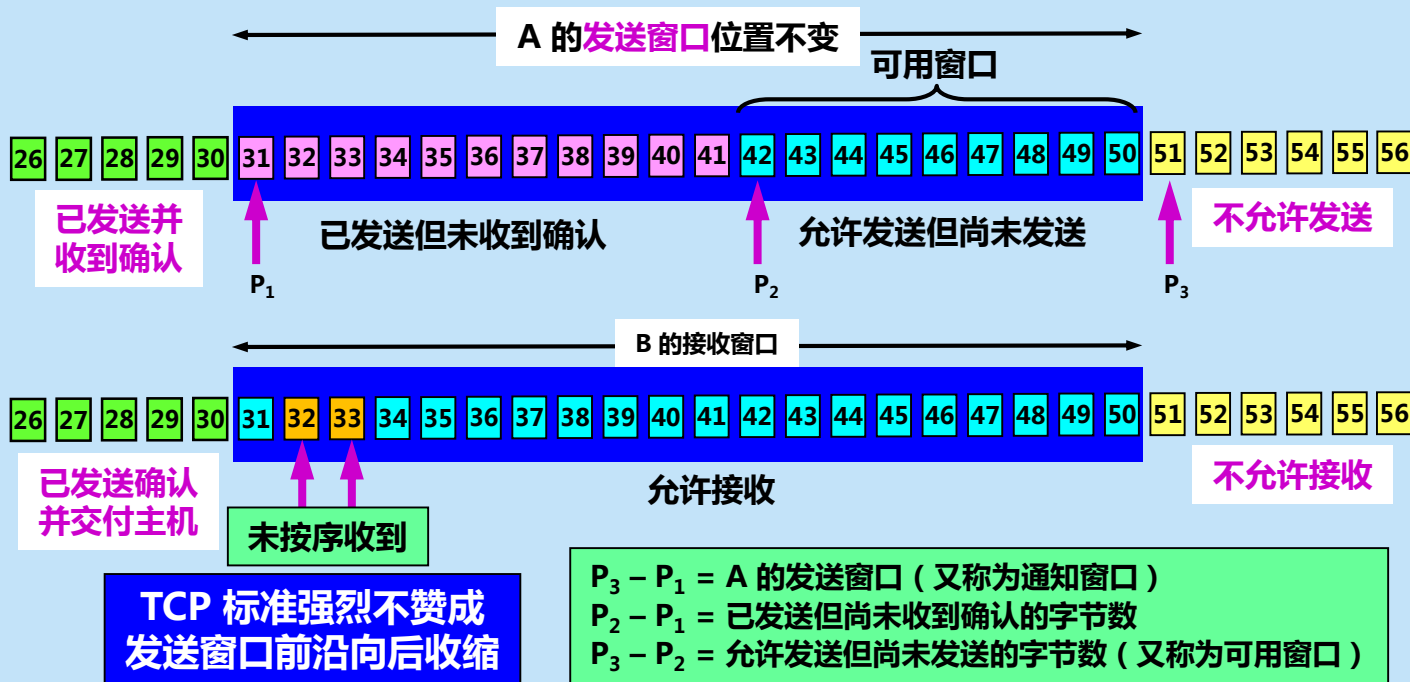


- 根据 B 给出的窗口值，A 构造出自己的发送窗口。
- **发送窗口表示**：在没有收到 B 的确认的情况下，A 可以连续把窗口内的数据都发送出去。
- 发送窗口里面的序号表示允许发送的序号。
- 显然，窗口越大，发送方就可以在收到对方确认之前连续发送更多的数据，因而可能获得更高的传输效率。



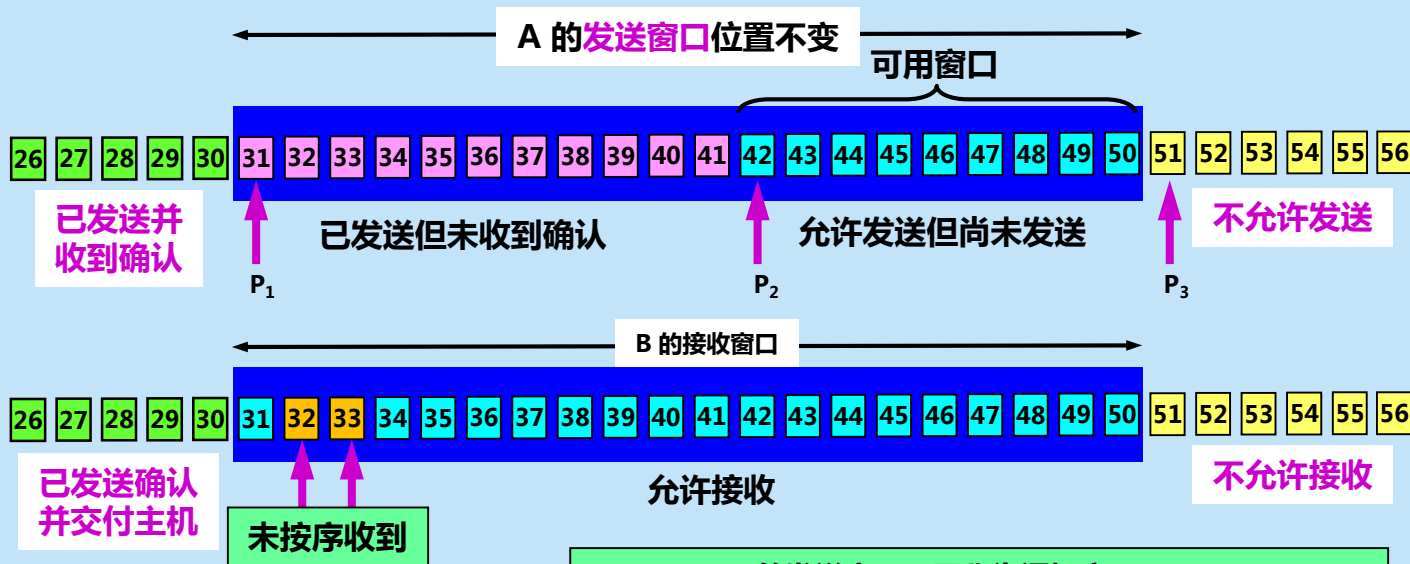


A 发送了 11 个字节的数据





A 发送了 11 个字节的数据

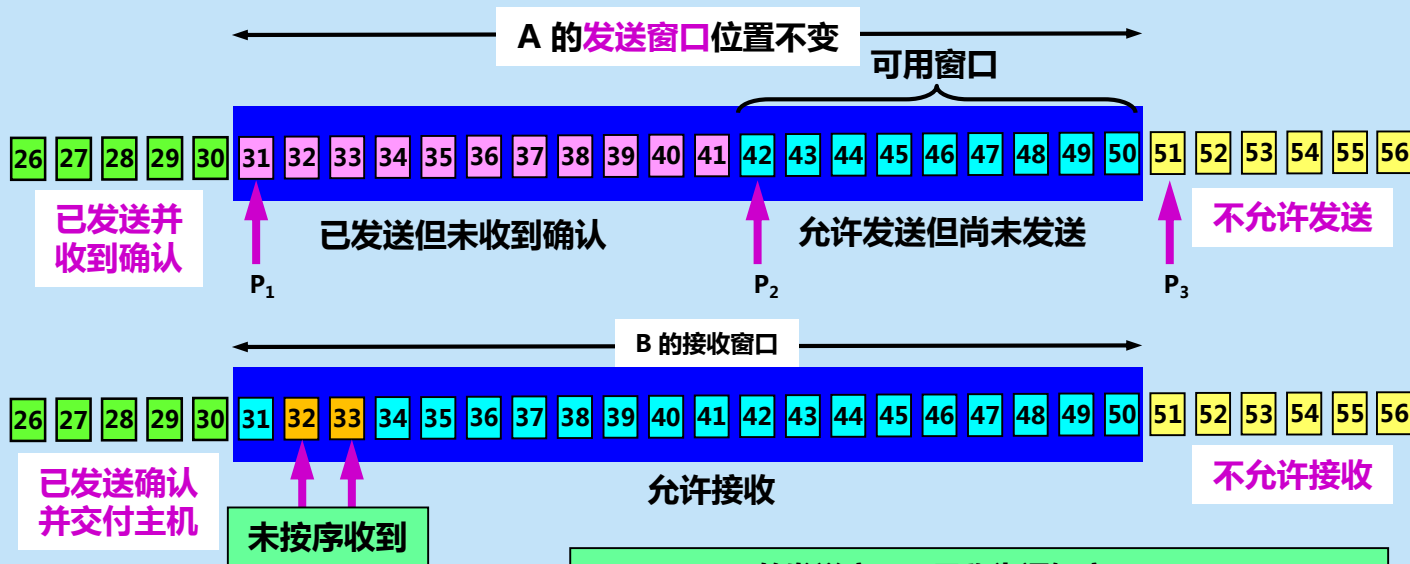


**TCP 标准强烈不赞成
发送窗口前沿向后收缩**

$P_3 - P_1 = A$ 的发送窗口 (又称为通知窗口)
 $P_2 - P_1 =$ 已发送但尚未收到确认的字节数
 $P_3 - P_2 =$ 允许发送但尚未发送的字节数 (又称为可用窗口)



A 发送了 11 个字节的数据

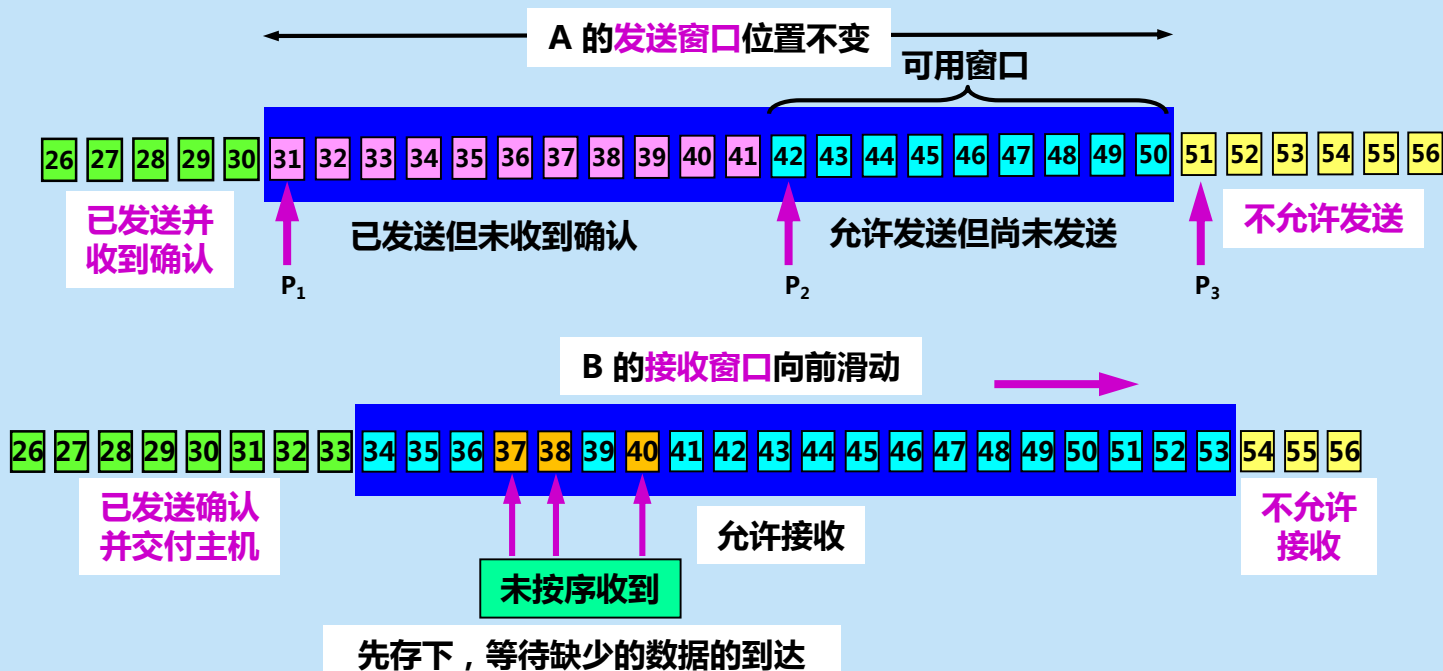


**TCP 标准强烈不赞成
发送窗口前沿向后收缩**

$P_3 - P_1 = A$ 的发送窗口 (又称为通知窗口)
 $P_2 - P_1 =$ 已发送但尚未收到确认的字节数
 $P_3 - P_2 =$ 允许发送但尚未发送的字节数 (又称为可用窗口)

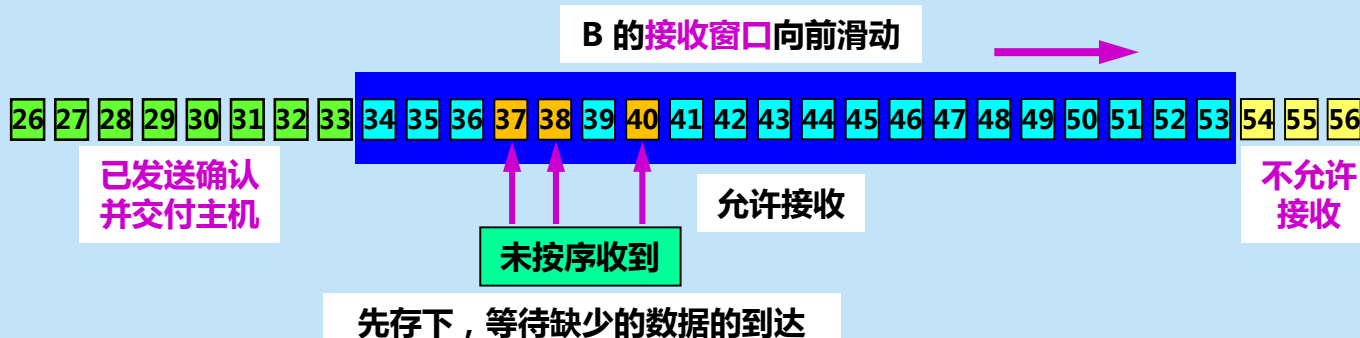
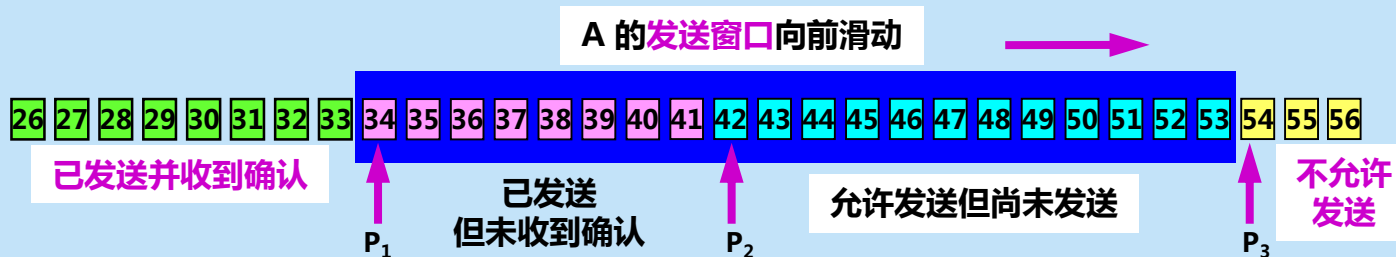


A 发送了 11 个字节的数据



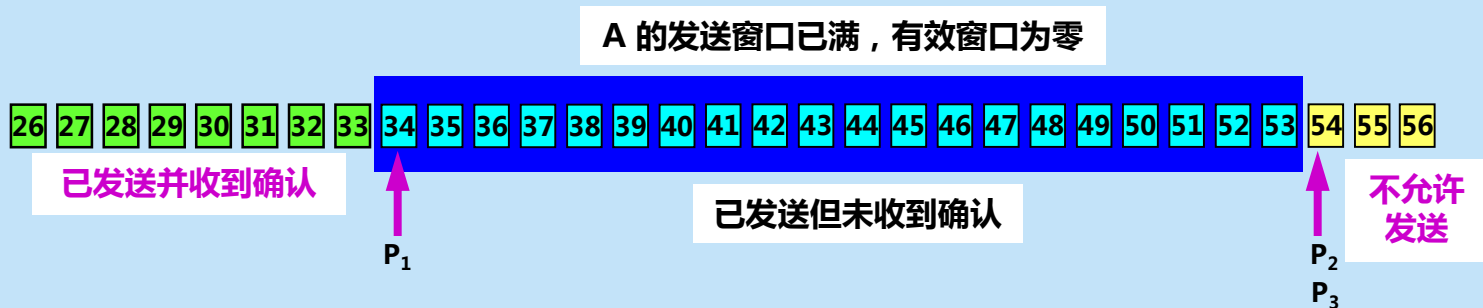


A 收到新的确认号，发送窗口向前滑动





**A 的发送窗口内的序号都已用完，
但还没有再收到确认，必须停止发送。**

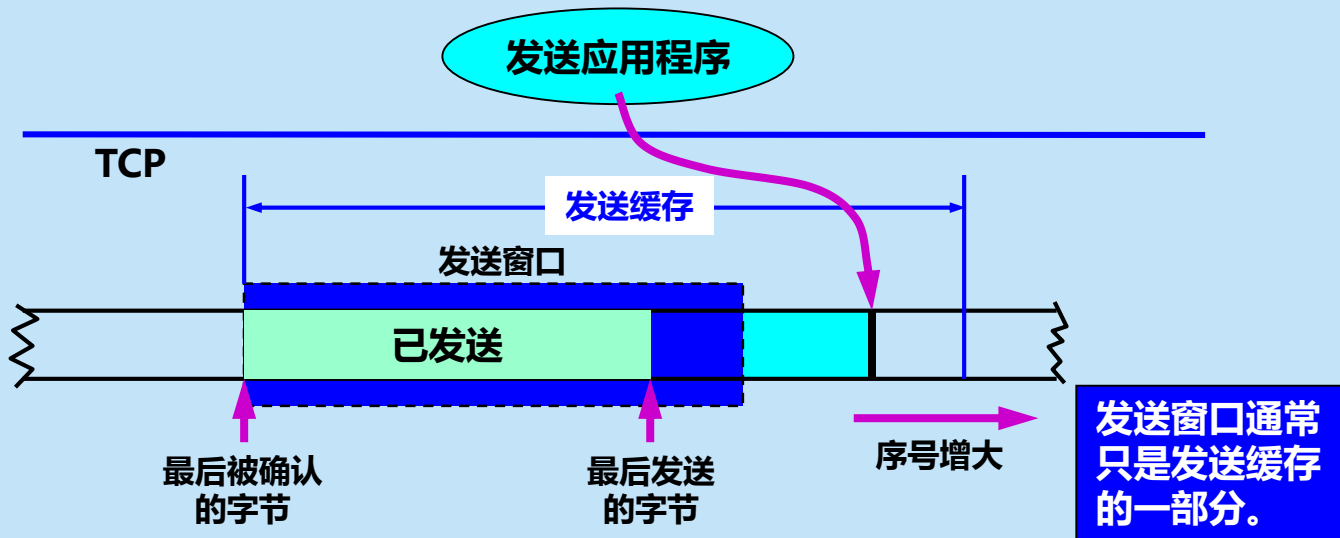


发送窗口内的序号都属于已发送但未被确认



发送缓存

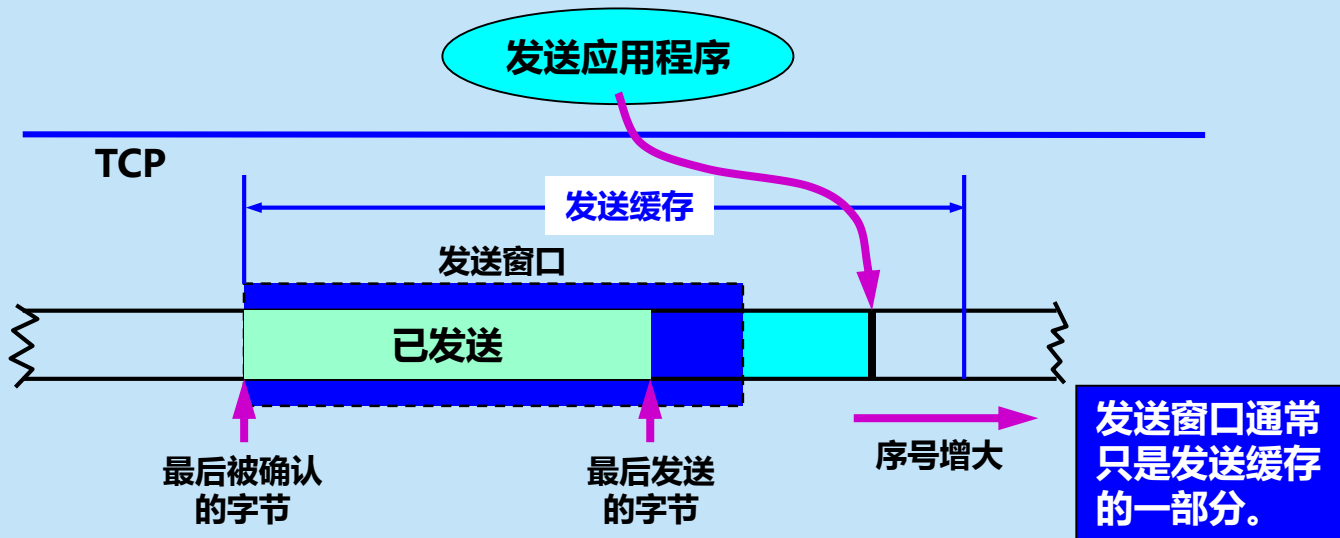
发送方的应用进程把字节流写入 TCP 的发送缓存。





发送缓存

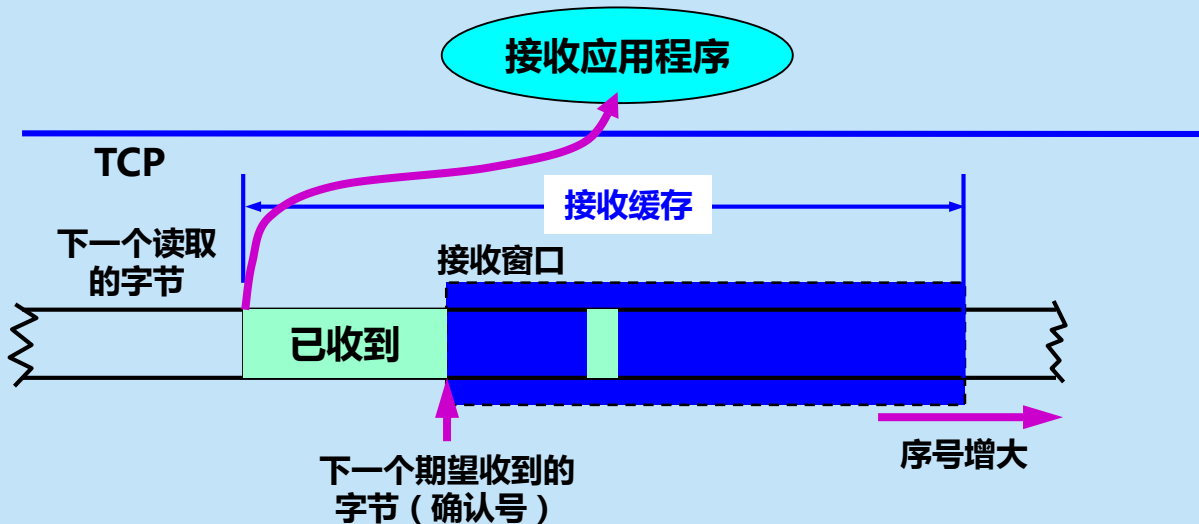
发送方的应用进程把字节流写入 TCP 的发送缓存。





接收缓存

接收方的应用进程从 TCP 的接收缓存中读取字节流。





发送缓存与接收缓存的作用

- **发送缓存**用来暂时存放：
 1. 发送应用程序传送给发送方 TCP 准备发送的数据；
 2. TCP 已发送出但尚未收到确认的数据。
- **接收缓存**用来暂时存放：
 1. 按序到达的、但尚未被接收应用程序读取的数据；
 2. 不按序到达的数据。



需要强调三点

- **第一**，A 的发送窗口并不总是和 B 的接收窗口一样大（因为有一定的时间滞后）。
- **第二**，TCP 标准没有规定对不按序到达的数据应如何处理。通常是先临时存放在接收窗口中，等到字节流中所缺少的字节收到后，再按序交付上层的应用进程。
- **第三**，TCP 要求接收方必须有累积确认的功能，这样可以减小传输开销。

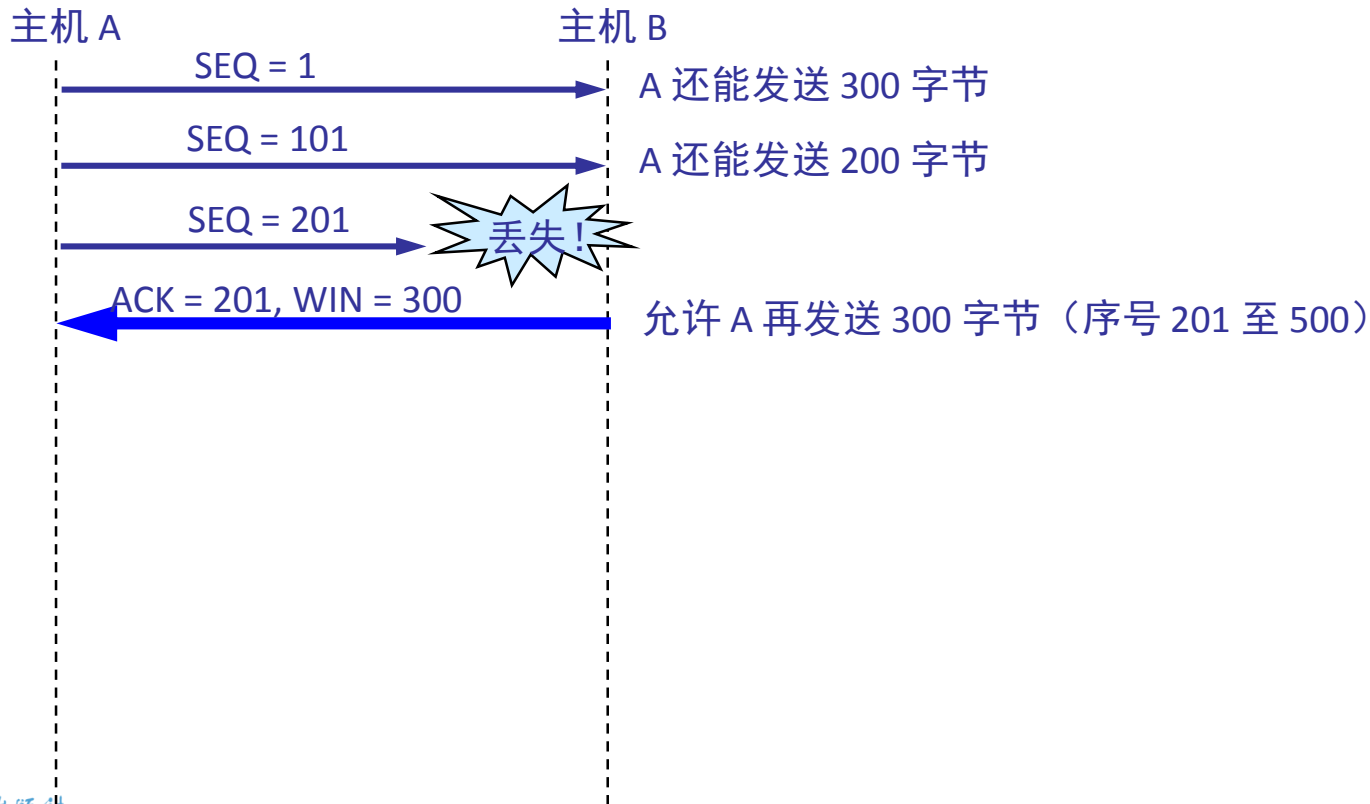


接收方发送确认

- 接收方可以在**合适的时候发送确认**，也可以在自己有数据要发送时把确认信息**顺便捎带上**。
- 但请注意两点：
 1. 第一，接收方不应过分推迟发送确认，否则会导致发送方不必要的重传，这反而浪费了网络的资源。。
 2. 第二，捎带确认实际上并不经常发生，因为大多数应用程序很少同时在两个方向上发送数据。

利用可变窗口大小进行流量控制

双方确定的窗口值是 400



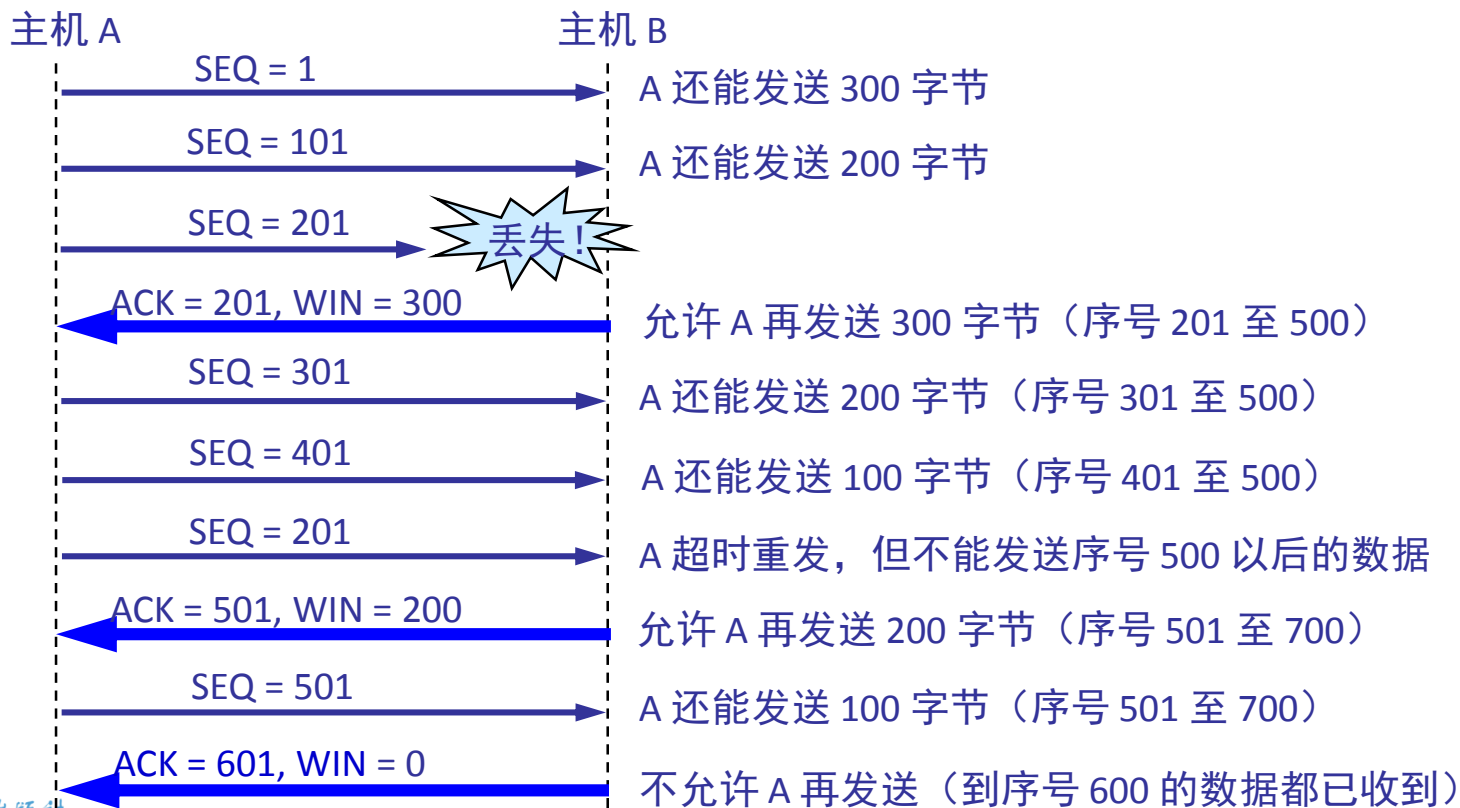
利用可变窗口大小进行流量控制

双方确定的窗口值是 400



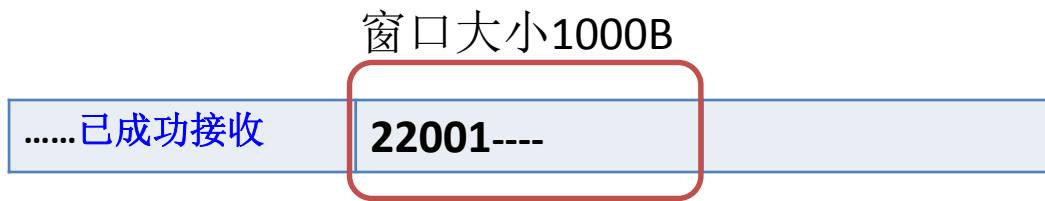
利用可变窗口大小进行流量控制

双方确定的窗口值是 400



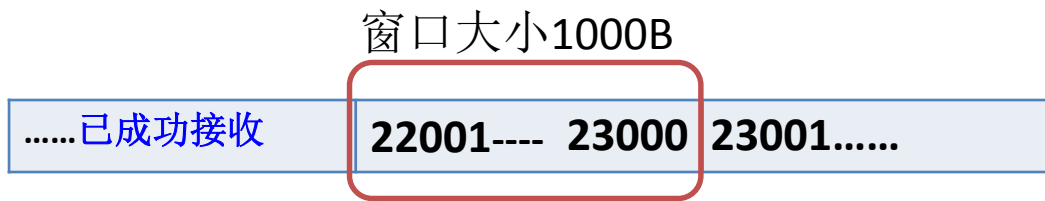


【例1】 TCP连接使用1000字节窗口值，上一次的确认号为22001，试问：现在收到了一个报文段，发出的确认号为22401，其窗口字段变为1200字节，请图示说明这之前与之后的窗口变化。



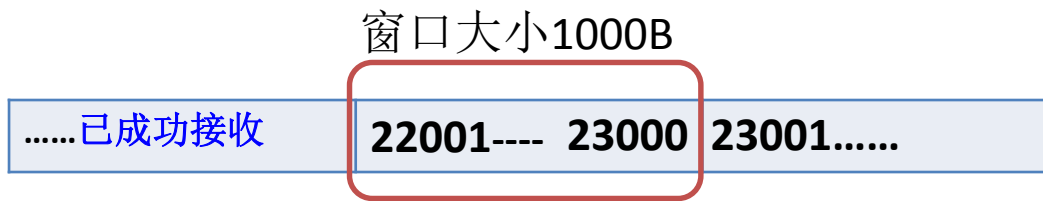


【例1】 TCP连接使用1000字节窗口值，上一次的确认号为22001，试问：现在收到了一个报文段，发出的确认号为22401，其窗口字段变为1200字节，请图示说明这之前与之后的窗口变化。



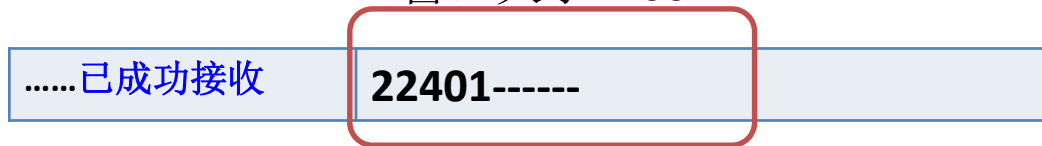


【例1】 TCP连接使用1000字节窗口值，上一次的确认号为22001，试问：现在收到了一个报文段，发出的确认号为22401，其窗口字段变为1200字节，请图示说明这之前与之后的窗口变化。



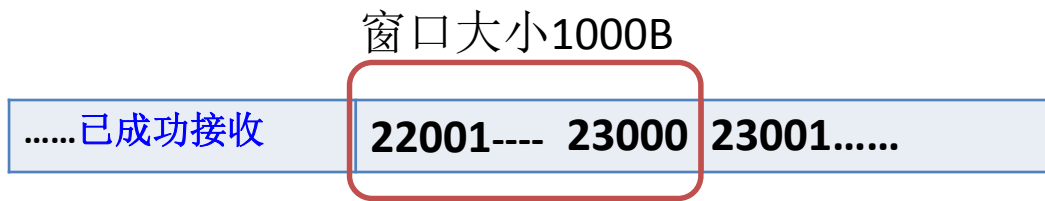
窗口大小1200B

正确接收了400字节





【例1】 TCP连接使用1000字节窗口值，上一次的确认号为22001，试问：现在收到了一个报文段，发出的确认号为22401，其窗口字段变为1200字节，请图示说明这之前与之后的窗口变化。





【例2】 主机甲与主机乙之间已建立一个TCP连接，双方持续有数据传输，且数据无差错与丢失。

若甲收到1个来自乙的TCP段，该段的序号为1913、确认序号为2046、有效载荷为100字节，则甲立即发送给乙的TCP段的序号和确认序号分别是多少？

- A. 2046、2012
- B. 2046、2013
- C. 2047、2012
- D. 2047、2013



【例3】 主机A向主机B连续发送了两个TCP报文段，其序号分别为70和100，试问：

- ① 第一个报文段携带了多少字节数据？
- ② 主机B收到第一个报文段后发回的确认中的确认号是多少？
- ③ 如果B收到第二个报文段后发回的确认号是280，试问A发送的第二个报文段中的数据有多少字节？
- ④ 如果A发送的第一报文段丢失了，但第二个报文段到达了B。B在第二个报文段到达后向A发送确认，这个确认号是多少？



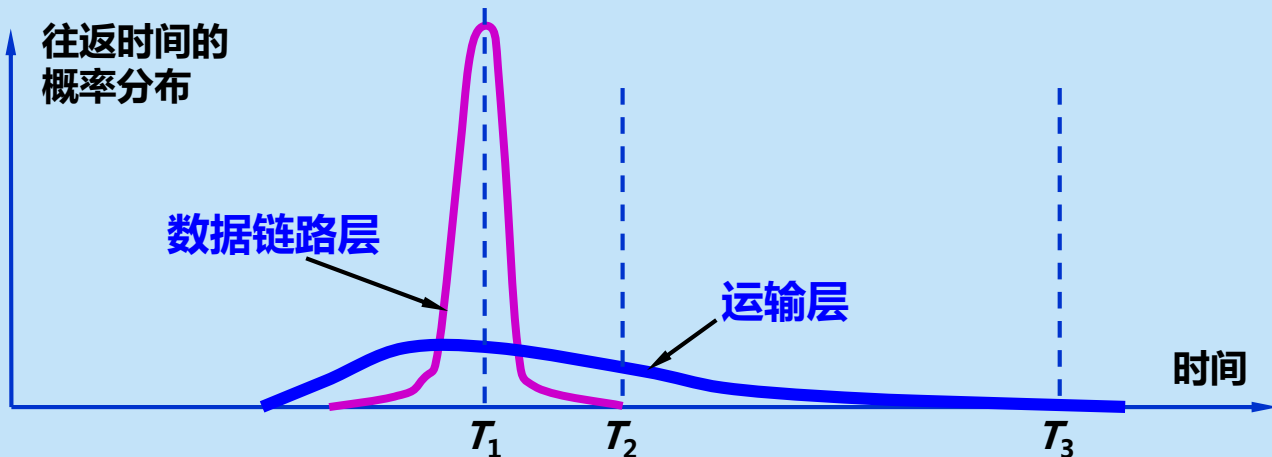
5.6.2 超时重传时间的选择

- 重传机制是 TCP 中最重要和最复杂的问题之一。
- TCP 每发送一个报文段，就对这个报文段设置一次计时器。
- 只要计时器设置的重传时间到但还没有收到确认，就要重传这一报文段。
- 重传时间的选择是 TCP 最复杂的问题之一。



往返时延的方差很大

由于 TCP 的下层是一个互联网环境，IP 数据报所选择的路由变化很大。因而运输层的往返时间 (RTT) 的方差也很大。





TCP 超时重传时间设置

- 如果把超时重传时间设置得太短，就会引起很多报文段的不必要的重传，使网络负荷增大。
- 但若把超时重传时间设置得过长，则又使网络的空闲时间增大，降低了传输效率。
- **TCP 采用了一种自适应算法**，它记录一个报文段发出的时间，以及收到相应的确认的时间。这两个时间之差就是报文段的往返时间 RTT。



加权平均往返时间

- TCP保留了RTT的一个**加权平均往返时间** RTT_S (这又称为**平滑的往返时间**)。
- 第一次测量到 RTT 样本时, RTT_S 值就取为所测量到的 RTT 样本值。以后每测量到一个新的 RTT 样本, 就按下式重新计算一次 RTT_S :

$$\text{新的}RTT_S = (1 - \alpha) \times (\text{旧的}RTT_S) + \alpha \times (\text{新的}RTT\text{样本}) \quad (5-4)$$

- 式中, $0 \leq \alpha < 1$ 。若 α 很接近于零, 表示 RTT 值更新较慢。若选择 α 接近于 1, 则表示 RTT 值更新较快。
- RFC 6298 推荐的 α 值为 $1/8$, 即 0.125。



超时重传时间 RTO

- RTO (Retransmission Time-Out) 应略大于上面得出的加权平均往返时间 RTT_S 。
- RFC 6298 建议使用下式计算 RTO :

$$RTO = RTT_S + 4 \times RTT_D \quad (5-5)$$

- RTT_D 是 RTT 的偏差的加权平均值。
- RFC 6298 建议这样计算 RTT_D 。第一次测量时, RTT_D 值取为测量到的 RTT 样本值的一半。在以后的测量中, 则使用下式计算加权平均的 RTT_D :

$$\text{新的 } RTT_D = (1 - \beta) \times (\text{旧的 } RTT_D) + \beta \times |RTT_S - \text{新的 } RTT \text{ 样本}| \quad (5-6)$$

- β 是个小于 1 的系数, 其推荐值是 1/4, 即 0.25。



超时重传时间 RTO

$$RTO = RTT_S + 4 \times RTT_D$$

$$\text{新的} = (1 - 0.25) \times (\text{旧的} RTT_D) + 0.25 \times |RTT_S - \text{新的} RTT \text{ 样本}|$$

如果忽略偏差值 $|RTT_S - \text{新的} RTT \text{ 样本}|$ ，那么可以近似得出：

$$RTT_{D1} = 1/2 RTT_1, \quad RTO_1 = RTT_S + 2 RTT_1$$

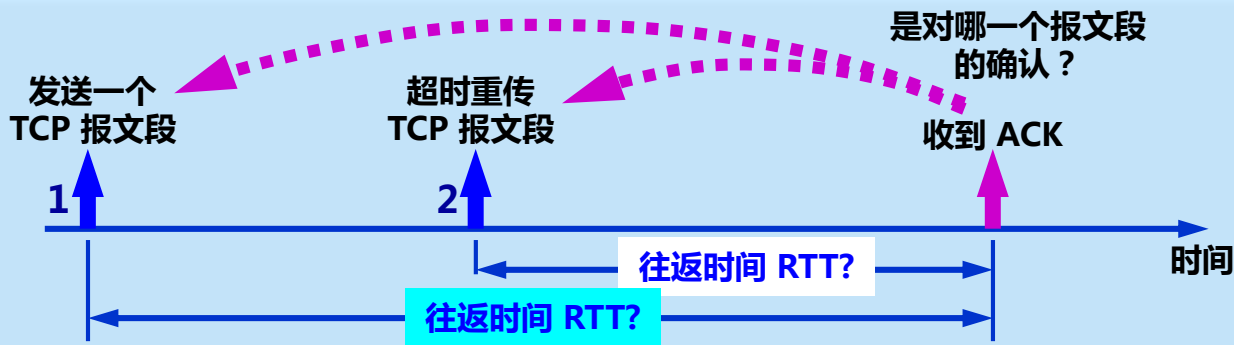
$$RTT_{D2} = 3/8 RTT_1, \quad RTO_2 = RTT_S + 3/2 RTT_1$$

$$RTT_{D3} = 9/32 RTT_1, \quad RTO_3 = RTT_S + 9/8 RTT_1$$



往返时间 (RTT) 的测量相当复杂

- TCP 报文段 1 没有收到确认。重传 (即报文段 2) 后, 收到了确认报文段 ACK。
- 如何判定此确认报文段是对原来的报文段 1 的确认, 还是对重传的报文段 2 的确认?





Karn 算法

- 在计算平均往返时间 RTT 时，只要报文段重传了，就不采用其往返时间样本。这样得出的加权平均平均往返时间 RTT_s 和超时重传时间 RTO 就较准确。
- 但是，这又引起新的问题。当报文段的时延突然增大了很多时，在原来得出的重传时间内，不会收到确认报文段。于是就重传报文段。但根据 Karn 算法，不考虑重传的报文段的往返时间样本。这样，超时重传时间就无法更新。



修正的 Karn 算法

- 报文段每重传一次，就把 RTO 增大一些：

$$\text{新的 RTO} = \gamma \times (\text{旧的 RTO})$$

- 系数 γ 的典型值是 2。
- 当不再发生报文段的重传时，才根据报文段的往返时延更新平均往返时延 RTT 和超时重传时间 RTO 的数值。
- 实践证明，这种策略较为合理。



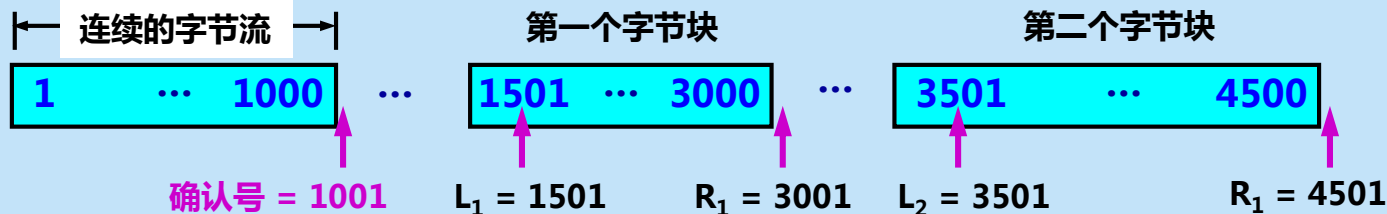
5.6.3 选择确认 SACK

- **问题**：若收到的报文段无差错，只是未按序号，中间还缺少一些序号的数据，那么能否设法只传送缺少的数据而不重传已经正确到达接收方的数据？
- **答案是可以的。选择确认 SACK (Selective ACK) 就是一种可行的处理方法。**



接收到的字节流序号不连续

TCP 的接收方在接收对方发送过来的数据字节流的序号不连续，结果就形成了一些不连续的字节块。



和前后字节不连续的每一个字节块都有两个边界：边界和右边界。

- 第一个字节块的左边界 $L_1 = 1501$ ，但右边界 $R_1 = 3001$ 。左边界指出字节块的第一个字节的序号，但右边界减 1 才是字节块中的最后一个序号。
- 第二个字节块的左边界 $L_2 = 3501$ ，而右边界 $R_2 = 4501$ 。



RFC 2018 的规定

- 如果要使用选择确认，那么在建立 TCP 连接时，就要在 TCP 首部的选项中加上“允许 SACK”的选项，而双方必须都**事先商定好**。
- 如果使用选择确认，那么原来首部中的“确认号字段”的用法仍然不变。只是以后在 TCP 报文段的**首部中都增加了 SACK 选项**，以便报告收到的不连续的字节块的边界。
- 由于首部选项的长度最多只有 40 字节，而指明一个边界就要用掉 4 字节，因此在**选项中最多只能指明 4 个字节块的边界信息**。